

Implementation of selected functionalities in the client-side part of the Accreditation application

Master's Thesis

Bc. Marta Steklá

Supervisor:
Ing. Oldřich Faldík, PhD.

Brno 2026



DIPLOMA THESIS TOPIC

Author of thesis: Bc. Marta Steklá
Study programme: Open Informatics
Scope: Scope for Open Informatics
Thesis supervisor: Ing. Oldřich Faldík, Ph.D.

Title of the thesis: **Implementation of selected functionalities in the client-side part of the Accreditation application**

Language version: English

Guides to writing a thesis:

1. Analyze the current state of the process and the application for managing accreditation files at MENDELU. Furthermore, analyze the client's expanding requirements for selected parts of the client application. Focus mainly on the following areas: A-I and B-I forms, form statuses, form approval process, administration sub-applications and publication section within the user profile.
2. Design selected parts of the client application so that they are compatible with the application as a whole and with the Smart MENDELU platform, ensuring that it is user-friendly and secure, and that future client requirements can be implemented more easily. The application will be based on TypeScript and Vue.js technologies.
3. Implement and test the proposed solution. Include end-to-end tests as part of the testing process.
4. Discuss the resulting solution with respect to the current state and possibilities for further expansion.

Length of thesis: 2,5 – 4 AA

Bibliography:

1. ROJAS, Carlos. 2020. *Building Progressive Web Applications with Vue.js*. New York: Apress, 242 p. ISBN 978-1-4842-5334-2.
2. LISTWON, Benjamin; HANCHETT, Erik. 2019. *Vue.js in Action*. USA: Manning Publications, 375 p. ISBN 978-1-61729-462-4.
3. YOU, Evan. 2025. *Vue.js: The Progressive JavaScript Framework*. <https://vuejs.org>.
4. *Vue.js: up and running: building accessible and performant web apps*. 2018. Sebastopol: O'Reilly, 158 p. ISBN 978-1-491-99724-6.

Date of entry: December 2025

Date of submission: May 2026

Electronically approved: 01/16/2026
Ing. Oldřich Faldík, Ph.D.
Thesis supervisor

Electronically approved: 01/26/2026
doc. Ing. Oldřich Trenz, Ph.D.
Head of Department

Electronically approved: 01/27/2026
doc. Ing. František Dařena, Ph.D.
Study programme supervisor

Electronically approved: 01/28/2026
Bc. Marta Steklá
Author of thesis

I would like to express my sincere gratitude to my supervisor, Ing. Oldřich Faldík, PhD., for his professional guidance and valuable advice during the preparation of this thesis. Special thanks go to the Smart MENDELU team for the opportunity to help develop the Accreditation application, working with such a talented group was a great learning experience. Finally, I wish to thank my family and friends for their constant support and encouragement throughout my studies.

Declaration

I confirm by submitting that this thesis entitled *Implementation of selected functionalities in the client-side part of the Accreditation application* was written and completed by me. I also declare that all the sources and information used to complete the thesis are included in the list of references. I agree that the thesis could be made public in accordance with Article 47b of Act No. 111/1998 Coll., on Higher Education Institutions and on Amendments and Supplements to Some Other Acts (the Higher Education Act), and in accordance with the current Directive on publishing of the final theses. I am aware that my thesis is written in accordance to Act. No. 121/2000 Coll., on Copyright, and therefore Mendel University in Brno has the right to conclude licence agreements on the utilization of the thesis as a school work in accordance with Article 60 (1) of the Copyright Act.

Before concluding a licence agreement on utilization of the work by another person, I will request a written statement from the university that the licence agreement is not in contradiction to legitimate interests of the university, and I will also pay a prospective fee to cover the cost incurred in creating the work to the full amount of such costs.

Brno May 11, 2026

.....
signature

Contents

1	Introduction	11
1.1	Motivation and context	11
1.2	Problem statement	12
1.3	Objectives and thesis goals	13
2	Digitalization	14
2.1	Why do we digitalize?	14
2.2	Benefits of digitization	15
2.3	Trends in digitization	15
2.4	Best practices in digitalization	16
2.5	Similar projects undergoing digitalization	16
2.6	Digitalization at MENDEL University	17
2.6.1	Flagship Projects	18
3	Accreditation process	19
3.1	Legal basis and regulations	19
3.2	Quality control and the Evaluation board	20
3.3	The accreditation file and forms	21
3.4	Approval process	21
3.5	Current state of the accreditation process at MENDEL	22
4	Requirements	24
4.1	Main stakeholders	24
4.2	Overview of requirements	25
4.3	Functional requirements	25
4.4	Non-functional requirements	27
4.5	Milestones and planning of development	27
5	Frameworks and tools used in development	29
5.1	Vue.js	30
5.1.1	Use cases and best practices	31
5.1.2	Comparison to similar tools	31
5.2	JavaScript	31

5.2.1	Use cases and best practices	32
5.2.2	Comparison to TypeScript and migration analysis	32
5.3	ESLint	33
5.4	Vite	34
5.5	Scrumban	34
5.5.1	Trello	35
5.6	Git and Fork	36
5.7	SonarQube and Swagger	36
5.8	Docker	37
6	Design specifications	38
6.1	User interface and experience architecture	38
6.2	Component-based design approach	39
6.3	State and data flow design	40
6.4	Security and access control design	41
7	Implementation	42
7.1	Publications in User profile	42
7.1.1	Data model and user logic	43
7.1.2	Technical implementation	45
7.2	Form A-I	45
7.2.1	Core data fields	46
7.2.2	Architecture	46
7.2.3	Key dependencies and connections	48
7.2.4	Workflow Lifecycle	49
7.3	Form B-I	49
7.3.1	Core architecture	50
7.3.2	Functional sections of the form	50
7.4	Form status	52
7.4.1	Form status backend implementation	54
7.4.2	Form status implementation on front-end	56
7.5	Form approval process	58
7.5.1	Transition-based authority	58
7.5.2	The lifecycle of an approval workflow	59
7.5.3	Technical architecture of the approval engine	60
7.5.4	Communication and feedback loops	61
7.5.5	File-level overrides and finalization	62
7.6	Administration sub-application	63
7.6.1	Purpose and role of the Administration module	63
7.6.2	Standardized component architecture	65
7.6.3	Architectural design and component reusability	69

7.6.4	Data consistency and integrity	70
7.6.5	Integration with the main Accreditation workflow	71
8	Testing	72
8.1	User testing	72
8.2	Front-end testing levels	73
8.3	End-to-end testing with Cypress	73
8.3.1	Implementation of automated test scenarios	74
9	Conclusion	76
References		78
List of Tables		81
List of Figures		82

1 Introduction

Modern universities are increasingly transitioning their administrative processes to digital environments to manage growing volumes of data while maintaining high academic standards. As Khotimah et al. (2024) observe, shifting away from manual, fragmented systems toward integrated digital platforms is no longer merely an option, it is a necessity for institutions that wish to remain competitive and transparent. At Mendel University in Brno, one of the primary agendas currently undergoing digitalization is the accreditation of study programs. This complex process requires the collection and verification of extensive datasets ranging from teacher qualifications to detailed course syllabi to ensure compliance with both internal university standards and national regulations (Mendel University, 2024).

Historically, Mendel University handled these requirements through manual data entry into disconnected text documents and spreadsheets. Veverka (2025) notes that this reliance on manual labor created a heavy administrative burden for academic staff and left significant room for human error. To address these challenges, this thesis focuses on the development of specific client-side functionalities for a new, unified accreditation application. By utilizing the Vue.js framework and JavaScript, the project replaces outdated manual workflows with an intuitive user interface that automatically retrieves data from existing university systems (Karásek, 2025). The scope of this work encompasses the creation of core forms, a centralized approval engine, and administrative tools designed to maintain data integrity throughout the entire accreditation lifecycle.

1.1 Motivation and context

The primary motivation for this project stems from the inefficiencies inherent in the university's current approach to accreditation management. Relying on static templates for such complex documentation means that stakeholders cannot track the progress of an application in real time, which often leads to administrative bottlenecks. Statistics from OECD (2023) suggest that manual data handling can result in error rates of approximately 8 percent, whereas integrated digital systems significantly increase accuracy. A major objective of this development is to eliminate repetitive data entry by pulling information directly from the University Information System (UIS). As Veverka (2025) emphasizes, this integration ensures that every accreditation file is built upon

the most recent and verified institutional records.

This initiative is part of a broader digitalization strategy at Mendel University. The transition became particularly urgent following the COVID-19 pandemic, which exposed the vulnerabilities of traditional, on-site processes and accelerated the demand for robust digital infrastructure (Hou et al., 2022). Furthermore, from a legal perspective, the application must comply with the Higher Education Act and the university's own internal quality regulations. These frameworks require strict oversight and a specific approval path involving the Council for Internal Evaluation (RVH) (Mendel University, 2023). The new application serves as a central vehicle for transforming these complex legal and administrative requirements into a clear, manageable digital workflow.

1.2 Problem statement

At present, managing accreditation at the university remains a scattered and largely manual task. Faculty members must construct complicated dossiers using basic text editors and spreadsheets, which forces them to repeatedly copy data from the UIS. This duplication of effort not only wastes valuable time but also leads to frequent typographical errors. Because these dossiers are stored as individual, offline files, there is no effective mechanism to keep them synchronized with the live data stored in the university's primary databases (Veverka, 2025).

These difficulties persist throughout the evaluation and approval phases. Currently, bodies such as the RVH lack a centralized platform to monitor how an application is progressing through the hierarchy and identifying the status of critical forms typically requires searching through extensive email chains or requesting manual updates from faculty representatives. This lack of transparency complicates quality monitoring and slows the entire approval process for new and existing study programs (Karásek, 2025).

From a technical standpoint, the absence of a modern web interface means that the university lacks the tools necessary to perform automatic error checking during data entry as static documents cannot provide the dynamic feedback or structured inputs required for high-quality institutional reporting. This technical gap hinders the university from achieving its strategic goal of full digitalization (Chounta et al., 2024). Therefore, developing a dedicated web interface is essential to bridge the gap between raw institutional data and the sophisticated reporting required for modern academic accreditation (Mendel University, 2024).

1.3 Objectives and thesis goals

The principal goal of this thesis is to implement specific client-side components of the Mendel University accreditation system. The project centers on the creation of a web interface that is easy to navigate while remaining capable of handling the university's complex administrative requirements. By building the application with Vue.js and JavaScript, the project establishes a robust technical foundation that is both easier to maintain and less prone to common development errors.

A core task involves the development of specialized modules for the A-I and B-I forms. These modules are designed to retrieve academic data from the UIS automatically, thereby minimizing the need for manual input. Beyond simple data entry, the application provides a comprehensive system for tracking the progression of files and managing the approval hierarchy. This provides all involved parties from faculty members to the RVH with a clear and immediate view of any file's current status.

Additionally, the project includes the development of an administration sub-application and a dedicated profile section for managing academic research publications. Each feature is designed to adhere to high security standards and prioritize ease of use for university staff. Finally, the thesis includes a rigorous testing phase to ensure that the system functions correctly under real-world conditions. This ensures that the final product meets the university's immediate needs while remaining extensible enough to support future expansions as the institution's requirements evolve.

2 Digitalization

This chapter explores the broader context of digital transformation in higher education and the strategic drivers behind the shift toward digital quality assurance. It examines the catalysts that necessitate this transition ranging from global events like the pandemic to national regulatory frameworks, while detailing the significant benefits in efficiency and accuracy that these systems provide. By reviewing current trends and best practices, the chapter establishes the theoretical and strategic foundation for the modernization efforts currently underway at Mendel University.

2.1 Why do we digitalize?

The shift toward digital quality assurance is primarily driven by strategic institutional needs. As Khotimah (2024) explains, universities must improve their efficiency and transparency to remain competitive in a modern landscape. By automating manual tasks, institutions can establish a more precise record of their decision-making processes, which in turn allows them to hold departments more accountable for their overall performance.

While strategic goals provided the initial motivation, the COVID-19 pandemic served as a major catalyst for this transition. Hou et al. (2022) observe that the sudden impossibility of traditional on-site evaluations exposed a widespread lack of resilient digital infrastructure. Consequently, higher education quality assurance bodies were forced to quickly adopt remote evidence-gathering tools to maintain the integrity of the accreditation process during global lockdowns.

In addition to these internal and external pressures, government-led initiatives have played a critical role. For instance, the OECD (2023) highlights that in countries like Hungary, national policies emphasize that digitalization is no longer optional but a fundamental requirement for high-quality service delivery. These frameworks encourage institutions to build sustainable digital systems that align with specific regulatory expectations.

2.2 Benefits of digitization

One of the most immediate advantages of a digital accreditation system is the substantial time savings it offers. Setiyadi et al. (2025) note that implementing digital workflows can reduce administrative task time by up to 70%. Their research highlights specific cases where document processing speed improved by 75%, dropping from 180 seconds down to just 45 seconds per task.

Beyond speed, accuracy also sees a significant boost when human data entry is replaced by automation. Setiyadi et al. (2025) further demonstrate that digital platforms can reduce data-handling errors by 30%, often raising accuracy rates from 92% to a near-perfect 99%. Such precision is vital for maintaining the validity of complex accreditation files.

The transition also provides stakeholders with better visibility through real-time analytics. Rather than relying on delayed manual reports, Khotimah (2024) suggests that administrators can now access shared metrics the moment data is entered. This transparency allows for much faster intervention if any part of the accreditation process begins to fall behind schedule.

Finally, digitalization fundamentally changes how stakeholders interact with quality assurance. Vanhear et al. (2025) argue that collaborative platforms encourage deeper participation from faculty and peer reviewers. Using Malta's peer-review system as an example, they show how digital tools can transform a process from a simple compliance exercise into a meaningful, active collaboration.

2.3 Trends in digitization

There is a growing movement toward fully electronic models for internal quality assessment. Mostafavi et al. (2026) explain that these systems go beyond simply digitizing paper records; they require a complete rethinking of organizational structures and ICT support. Their study of electronic universities suggests that for a digital approach to be truly effective, it must encompass every dimension of the institution.

Before adopting such systems, however, many institutions are now using data-informed readiness frameworks. Chounta et al. (2024) describe this as a trend where universities use multi-dimensional indicators to measure their capacity for change. This proactive assessment helps identify where additional training or technical infrastructure is needed before a project officially begins.

Looking toward the future, artificial intelligence is starting to reshape how

accreditation is handled. Ilieva et al. (2025) and Isaifan (2026) both point to the integration of generative AI and decision-support tools within assessment frameworks. These technologies are expected to reform global practices by allowing evaluators to process vast amounts of data more consistently while providing faster feedback to the institutions.

2.4 Best practices in digitalization

A successful digital transformation often requires a structured management approach, such as the Plan-Do-Check-Act (PDCA) cycle. As Zakharova (2025) highlights, this method ensures that digitalization is treated as a continuous process of improvement rather than a one-time technical event, helping to keep development aligned with the evolving needs of university stakeholders.

During the design phase, Ramezani et al. (2025) suggest using comprehensive accreditation checklists that cover organizational, technological, and pedagogical criteria. By integrating these various perspectives early on, the institution ensures that the new system supports all facets of its academic mission, not just the administrative ones.

From a technical standpoint, maintaining consistency is best achieved through standardized data processing. Setiyadi et al. (2025) recommend the use of Extract, Transform, and Load (ETL) techniques to ensure that digitized documents are clean and traceable. This is particularly useful for managing the diverse and often unstructured evidence required for accreditation.

Finally, the success of any implementation depends on co-creation with end users. Vanhear et al. (2025) emphasize that developers should work directly with students, faculty, and administrators to design workflows and rubrics. This collaborative focus ensures the final application is user-friendly and addresses the practical, daily needs of its users.

2.5 Similar projects undergoing digitalization

Practical examples of these systems can be seen at Surabaya State University, where a digital quality assurance platform successfully streamlined complex workflows. According to Khotimah (2024), the use of real-time dashboards allowed university leadership to make much more informed decisions regarding program quality and error reduction.

Another project focused specifically on a model for digitizing documentation

using the ETL method. Setiyadi et al. (2025) report that this platform accelerated the validation process for assessors, providing a more reliable way to manage the massive volume of documents generated during a typical accreditation cycle.

A pilot study of an electronic university quality assessment model similarly demonstrated the value of a multi-dimensional approach. Mostafavi et al. (2026) found that when organizational and technical support systems are well-integrated, they can effectively manage the complexities of university-wide assessment while maintaining high academic standards.

The University of Malta's digital peer-review platform further illustrates the benefits of flexible design. Vanhear et al. (2025) describe how their system utilizes no-code rubrics to allow for rapid adjustments and collaborative evaluation. This project proved that digital tools can significantly speed up decision-making without compromising the integrity of the peer-review process.

2.6 Digitalization at MENDEL University

Mendel University in Brno is currently pursuing a strategy of digital transformation to modernize its administrative and academic frameworks. As Karásek (2025) notes, this effort focuses on shifting away from paper-based workflows toward integrated digital solutions that enhance data accessibility for both staff and students. Rather than relying on generic third-party software, the university prioritizes the creation of internal tools tailored to its specific departmental needs.

The Section for the Electronization and Digitization of Agendas serves as the central unit overseeing these shifts. This department manages the lifecycle of university agendas, ensuring that complex institutional processes are transitioned smoothly into digital environments. They work in close coordination with the university's quality department to ensure that all software development remains in line with both regulatory requirements and institutional standards.

The technical development of these tools is led by SmartMENDEL, the brand and team behind the university's modern application ecosystem. Their work focuses on building user-friendly interfaces through a unified design system. By utilizing contemporary technologies such as Vue.js and TypeScript, they create extensible client-side applications designed to grow and adapt alongside the university's future requirements.

2.6.1 Flagship Projects

The university has launched several key applications that demonstrate the impact of its digitalization strategy:

- **Accreditation (Akreditace):** This system manages the complex process of creating and approving study program accreditation files, replacing manual document handling with a structured digital workflow.
- **Attendance (Docházka):** Developed as a mobile and web solution, this application allows MENDELU employees to record their working hours and specific activities.
- **MyMENDELU (Moje MENDELU):** This is the primary mobile application and web portal designed for the student body and general university community.

3 Accreditation process

This chapter provides a detailed overview of the regulatory and procedural framework that governs study programs in the Czech Republic. It outlines the legal basis established by national legislation and the specific internal rules of Mendel University, while also describing the complex structure of the accreditation file and the various bodies responsible for quality evaluation. Finally, the chapter analyzes the limitations of the current manual workflow to illustrate why a transition to a unified digital environment is necessary for the university's continued growth.

3.1 Legal basis and regulations

The accreditation of study programs in the Czech Republic is governed by the Higher Education Act (Act No. 111/1998 Sb), which establishes the legal conditions under which universities operate and grant academic degrees. This act also created the National Accreditation Bureau for Higher Education, which serves as the independent authority responsible for evaluating educational quality. As part of its mandate, the bureau decides on accreditation requests, ensures that national standards are followed, and maintains the official register of all accredited programs in the country.

Within this national framework, Mendel University operates according to its own internal guidelines, known as the Accreditation Rules of Mendel University. According to Mendel University (2023), this document is registered by the Ministry of Education, Youth and Sports and outlines the specific procedures for creating, approving, and modifying study programs. By defining the roles of individual university bodies and setting clear quality criteria, these rules provide a mandatory standard that all faculties and departments must follow to ensure academic excellence.

The university also benefits from holding institutional accreditation in several fields of study. As the National Accreditation Bureau (2024) explains, this status grants Mendel University a significant degree of autonomy, allowing the university to manage and approve its own programs without seeking individual permission for every change. However, this privilege requires the university to maintain a robust internal quality assurance system that mirrors national requirements, effectively shifting the responsibility for educational standards directly to the institution.

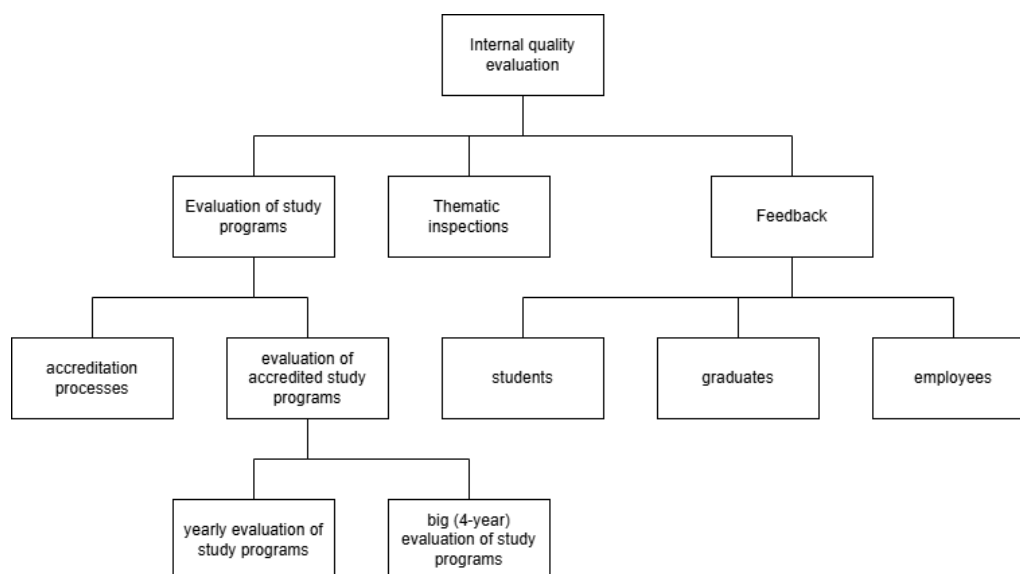


Figure 3.1

Quality evaluation at Mendel University (Mendel University, 2022)

3.2 Quality control and the Evaluation board

Quality control at Mendel University is treated as a continuous process aimed at monitoring and improving both educational and creative activities. According to Mendel University (2022), the system relies on regular internal evaluations where study programs are measured against specific quality indicators, such as staff qualifications, curriculum relevance, and graduate success. These findings allow the university to make data-driven decisions regarding the future of its academic offerings.

The primary body responsible for this oversight is the RVH. As described in the university's regulations (Mendel University, 2023), the council is composed of experts from various academic fields along with student representatives, ensuring a diverse perspective on institutional standards. The RVH acts as a gatekeeper, conducting the final internal review of study programs to ensure they meet the high standards set by both the university and national legislation before they can be launched or extended.

In addition to these internal checks, external quality control is maintained by the National Accreditation Bureau through periodic audits. While institutional accreditation covers many areas, any program falling outside that scope must undergo a direct evaluation by the national authority. Mendel University (2023) notes that this dual layer of control provides a comprehensive safety net, protecting the integrity of university degrees and ensuring the institution remains competitive on both a national and international level.

3.3 The accreditation file and forms

An accreditation file serves as a comprehensive collection of documents that describes every dimension of a study program. According to Mendel University (2023), this file acts as the formal evidence presented to evaluation bodies to prove that a program is well-designed and sufficiently supported. By using a standardized structure, the university ensures that reviewers can easily verify that all regulatory requirements have been addressed.

The core of this file is built upon standardized forms developed by the National Accreditation Bureau (2024). These include Form A-I, which captures basic details like the program title and degree type, and Form B-I, which provides a conceptual foundation by detailing the program's goals and intended learning outcomes. Together, these documents define the character of the accreditation request.

More technical details are captured in the B-II and B-III series of forms. While B-IIa outlines the study plans and sequences for bachelor and master programs, Form B-III focuses on individual subjects, listing specific topics, required literature, and the names of responsible teachers. As Mendel University (2023) points out, programs involving practical training must also include Form B-IV to describe the professional practice and the partner organizations involved.

Finally, the file includes the C-I personal forms for every faculty member involved in the program. These documents are critical for proving that the university possesses the necessary human resources, as they detail the academic qualifications, research history, and publishing records of the teaching staff. Collectively, these forms create a single, coherent digital or paper record that represents the complete study program.

3.4 Approval process

The approval process begins with the program's guarantor, who is responsible for drafting the initial proposal. According to Mendel University (2024), this involves collecting data for all A, B, and C forms and coordinating with the quality department to ensure compliance. Once the draft is complete, it undergoes a formal check to catch technical errors or missing information, a step that is vital for preventing delays later in the cycle.

Once the proposal passes the initial check, it is presented to the Dean's Board and the Faculty Scientific Board for an academic and strategic evaluation. If approved at the faculty level, the file moves to the university level for a deep

review by the RVH. At this stage, the council may approve the program, request revisions, or, in some cases, reject a proposal that fails to meet quality criteria.

The final internal approval is granted by the University Scientific Board. For programs under institutional accreditation, this decision is final, allowing the university to begin enrollment. However, Mendel University (2024) notes that if a program is not covered by institutional accreditation, the file must then be submitted to the National Accreditation Bureau for an external review and final approval.

3.5 Current state of the accreditation process at MENDELU

The current accreditation process at Mendel University still relies heavily on the manual collection and organization of data. As Veverka (2025) observes, program guarantors must manually gather details on faculty, course structures, and facilities, often recording them in Word and Excel templates. This preparation phase requires constant back-and-forth communication between authors and the quality department to ensure all forms meet the necessary standards.

Because there is no central platform, the process remains highly decentralized. Data is frequently extracted manually from the University Information System and re-entered into accreditation templates, a repetitive task that depends heavily on the diligence of individual staff members. While the final output is a massive, official accreditation file, the path to creating it is often fragmented and labor-intensive.

The approval of these files follows a strict hierarchy, moving from the faculty level to the University Scientific Board. At each stage, documents may be returned for revisions. As Mendel University (2022) points out, these feedback loops are typically handled through email or by exchanging different versions of text files, which can make it difficult to track the most recent changes or instructions.

This manual approach leads to a lack of transparency and a high risk of human error. Karásek (2025) notes that because information is scattered across different departments, it is nearly impossible for stakeholders to see the real-time status of a file. This often results in inconsistencies, such as a teacher's information being updated in one form but not another, which are frequently discovered late in the cycle, forcing authors to restart parts of the process.

The new application described in this thesis addresses these challenges by providing a unified environment for all accreditation activities within Mendel University. By integrating directly with existing university databases, the system pulls personnel and subject data automatically, eliminating manual entry

and preventing common errors. Furthermore, the built-in workflow manager tracks the progress of every file and sends automatic notifications to the relevant parties, ensuring that all participants work with the same data and can easily monitor the status of any application.

4 Requirements

This chapter identifies the core needs and technical expectations that guide the development of the new accreditation application. By analyzing the diverse roles of key stakeholders and establishing clear functional and non-functional requirements, it defines the essential scope of the technical solution. These requirements serve as the criteria for creating a secure, intuitive, and highly integrated environment designed to streamline all accreditation activities and administrative workflows at Mendel University.

4.1 Main stakeholders

The development of the accreditation system involves several groups with distinct roles and needs. According to Veverka (2025), the primary users are the faculty representatives, program guarantors, and academic staff. These individuals are responsible for the actual content of the accreditation files; consequently, they require a system that simplifies data entry and allows them to track the progress of their applications without navigating complex administrative hurdles. Their main objective is to ensure that every study program meets both the rigorous academic standards of the university and all applicable legal requirements.

Administrative oversight is provided by the Department of Quality and the RVH. While these stakeholders do not create the content themselves, Karásek (2025) notes that they manage the workflows and verify that all files comply with university regulations and national laws. To do this effectively, they require specialized tools for reviewing documents, providing feedback, and moving files through the various stages of the approval process. Furthermore, the Section for the Digitization of Agendas plays a key role by ensuring that the technical solution aligns with the broader digital strategy of Mendel University.

The technical development is carried out by the Smart MENDELU team. This group consists of developers and designers who focus on building a secure, user-friendly interface while ensuring the application remains compatible with existing university infrastructure. Finally, the university leadership acts as a strategic stakeholder. As Mendel University (2022) suggests, the leadership views the system as a vital tool for improving institutional efficiency and maintaining the high educational quality necessary for institutional accreditation.

4.2 Overview of requirements

The central requirement for the new system is to replace fragmented manual processes with a single, integrated digital environment. This involves creating a web-based application where all accreditation activities take place. Ideally, the system must support the entire lifecycle of an accreditation file, from the initial draft created by a faculty representative to the final approval granted by the university's scientific board. Transparency is a vital part of this requirement, as all participants need to see the real-time status of a file at any stage of the process.

Another critical objective is the integration of the system with existing university data sources. To reduce the administrative burden on academic staff, the application must pull information directly from the University Information System. This includes details about teaching staff, their publication records, and course descriptions. By automating this data retrieval, the system minimizes the risk of manual entry errors and ensures that accreditation files are always based on the most current records available at Mendel University.

The system also needs to provide a structured way to handle feedback and revisions. Instead of exchanging multiple versions of files through email, stakeholders require a centralized space for comments and corrections. This ensures that a complete history of the approval process is preserved and easily accessible. Finally, while the application must accommodate the specific needs of different faculty departments, it must maintain a consistent structure that meets the requirements of the National Accreditation Bureau, which may change or expand in the future.

4.3 Functional requirements

Functional requirements describe the specific actions the system must perform to satisfy user needs, essentially serving as a detailed roadmap of the software's features. Defining these clearly is a critical step, as it creates a formal agreement on what the final product will deliver to its users.

In this case, the system must provide robust tools for creating and managing core accreditation documentation. Users need to fill out and edit A-I and B-I forms directly within the application. Additionally, the system requires a dedicated publication section in the user profile. This module must integrate with the university's research database, allowing faculty members to easily select and link their academic works to specific accreditation files.

Table 4.1 Overview of functional and non-functional requirements.

Requirement category	Requirement type	Description
Core documentation	Functional	Electronic creation, editing, and management of A-I and B-I accreditation forms.
Publications module	Functional	Integration with the university research database to link academic works to user profiles.
Workflow management	Functional	Real-time tracking of file status and history throughout the university approval hierarchy.
System administration	Functional	Dedicated sub-application for managing system settings and assigning user roles.
Data integration	Functional	Automated retrieval of personnel and subject data from the University Information System (UIS).
User experience	Non-functional	Strict adherence to the Mendel University unified design system for a familiar interface.
Compatibility	Non-functional	Responsive web design ensuring full functionality across modern browsers and mobile devices.
Maintainability	Non-functional	Implementation using Vue.js and TypeScript to ensure a stable and extensible codebase.
Transparency	Non-functional	High visibility of the accreditation process for all involved stakeholders at any time.

The application must also manage the progression of files through the hierarchy of Mendel University. It is required to track the status of every document while maintaining a complete history of all changes and comments made during the review process. To support long-term management, the application includes an administration sub-application. This tool allows authorized users to manage

system settings and assign specific roles to participants. Furthermore, integration with the University Information System is mandatory to automatically populate teacher and subject data into the required forms.

4.4 Non-functional requirements

Non-functional requirements define the quality attributes and operational constraints of the system, focusing on how the system performs rather than specific tasks. These requirements such as speed, security, and ease of use, ensure that the application remains reliable and maintainable over the long term.

Usability is a central requirement for the client-side part of the application. To ensure the interface feels familiar and is easy to navigate, it must follow the unified design system of Mendel University. The goal is to create an intuitive experience where academic staff can complete their tasks with minimal technical support. Additionally, the system must be responsive and compatible with all modern web browsers to provide a consistent experience across various devices.

Security and maintainability are equally vital for the university's infrastructure. The application must utilize the central authentication service for user logins and implement role-based access control to protect sensitive information. To ensure the code can be easily updated or expanded, the development process relies on contemporary tools and technologies. These choices provide a robust structure that allows for the addition of new features without compromising the stability of the existing system.

4.5 Milestones and planning of development

The project began with a comprehensive analysis phase to map out existing processes and legal requirements. This included identifying the specific forms and workflows that needed to be digitized. Developers worked closely with the Department of Quality to ensure that every technical requirement matched the practical needs of Mendel University. This phase resulted in a detailed technical specification that served as a guide for implementing the client-side functionalities.

Implementation followed an iterative approach within the development framework of the Smart MENDELU team. The first major milestone was the design of the user interface and the development of the core forms. Once the

basic structure was established, the team focused on the approval logic and integration with university databases, allowing for early testing of the most critical features. Regular feedback sessions with stakeholders helped the team refine the interface and fix technical issues before moving to the next stage.

The final phase involves rigorous testing and preparation for deployment. This includes end-to-end tests to verify that the entire system works correctly from the user's perspective, alongside user testing with actual guarantors and administrative staff. The deployment is planned in stages to ensure a smooth transition from old manual methods to the new digital platform. Following the launch, a period of support and further expansion is planned to address evolving user needs within Mendel University.

5 Frameworks and tools used in development

Table 5.1 Overview of functional and non-functional requirements.

Tool / Technology	Category	Primary purpose
Vue.js	Frontend framework	Provides the reactive, component-based architecture for the user interface.
JavaScript	Programming language	Acts as the foundational logic layer for client-side interactivity.
TypeScript	Language superset	Adds static typing to catch errors early and improve IDE support.
ESLint	Static analysis	Enforces consistent coding styles and identifies potential syntax errors.
Vite	Build tool / Server	Serves the application during development with high-speed module replacement.
Scrumban	Methodology	Combines Scrum structure with Kanban flow for flexible project management.
Trello	Project management	Provides visual task tracking and transparency through a board-and-card system.
Git / GitLab	Version control	Manages source code history and facilitates collaboration between developers.
Fork	Git client	Offers a graphical interface to simplify complex version control operations.
SonarQube	Quality assurance	Scans for technical debt, security risks, and "code smells."
Swagger	API documentation	Provides interactive documentation for integrating frontend and backend logic.
Docker	Containerization	Ensures consistent execution environments across development and deployment.

This chapter provides a comprehensive technical overview of the technologies and methodologies employed to build the client-side part of the accreditation application. It details the selection of modern reactive frameworks and programming languages that ensure the system is both performant and maintainable, while also exploring the development servers and static analysis tools used to maintain high code quality. Furthermore, the chapter describes the agile management practices and collaborative tools that allowed the project to progress steadily within the specific organizational environment of Mendel University.

5.1 Vue.js

Vue.js is a progressive JavaScript framework designed for building modern user interfaces. As Tong et al. (2023) explain, the framework focuses primarily on the view layer and utilizes reactive data binding and component-based composition to manage complex UI logic. One of its defining characteristics is its incremental adoptability, which allows it to function as a simple script for minor enhancements or as the robust foundation for a full-scale single-page application.

The framework is currently moving toward a compiler-driven approach to maximize performance. A major trend in this evolution is the development of "Vapor Mode." Tong et al. (2023) describe this as an initiative to generate code that updates the real Document object model (DOM) directly, bypassing the virtual DOM to reduce memory usage and improve execution speed. Additionally, the ecosystem has shifted toward Vite as the standard build tool, which offers faster development starts than older tools like Webpack. Perdana et al. (2024) also point out that state management has transitioned from Vuex to Pinia, providing developers with better TypeScript support and a more intuitive API.

From a developer's perspective, Riyadi et al. (2024) highlight that Vue.js is highly regarded for its gentle learning curve, allowing those familiar with standard HTML and CSS to become productive quickly. However, they also note that its ecosystem is smaller than that of React, meaning there are fewer third-party libraries available. Furthermore, because it lacks the massive corporate backing of a company like Meta or Google, Tong et al. (2023) suggest that some large-scale enterprises perceive a higher risk regarding long-term support.

5.1.1 Use cases and best practices

This framework is an excellent choice for projects requiring rapid development and easy maintenance. Perdana et al. (2024) suggest it is particularly well-suited for administrative dashboards and internal university systems where high-quality user experience must be achieved with limited development resources. Because it is lightweight, it can also add interactivity to legacy websites without requiring a complete rewrite.

To ensure long-term maintainability, Riyadi et al. (2024) recommend using the Composition API for better logic organization in larger components. This approach allows related logic to be grouped together more effectively than the older Options API. They also suggest implementing "Script Setup" to reduce boilerplate code and using TypeScript to strictly type component props, which helps catch errors during development rather than at runtime.

5.1.2 Comparison to similar tools

Vue.js occupies a middle ground between the rigid structure of Angular and the flexibility of React. Unlike Angular, it does not require deep knowledge of complex patterns like dependency injection, making it more approachable for smaller teams. Compared to React, Tong et al. (2023) observe that Vue.js provides more built-in solutions for routing and state management, reducing the need for external libraries. Performance benchmarks typically show that Vue.js is faster than Angular and highly competitive with React for most common tasks.

5.2 JavaScript

JavaScript remains the foundational programming language of the web. As an interpreted, high-level language, it supports various programming paradigms, including object-oriented and functional styles. Tyler (2020) emphasizes that because it runs natively in all modern browsers, it is the most widely used technology for building digital interfaces and cross-platform applications.

The JavaScript ecosystem is currently seeing a shift toward performance optimization. According to Devographics (2026), the ECMAScript 2025 specification introduces features like the `Float16Array` for AI-driven tasks and lazy

evaluation in new Iterator objects. There is also a notable movement toward a "return to simplicity," where developers use lightweight tools like Alpine.js to add interactivity without the overhead of massive frameworks.

One of the primary advantages of JavaScript is its massive ecosystem. Raymond (2023) notes that the language's flexibility allows for rapid iteration and deployment, while its non-blocking I/O model makes it effective for real-time applications. However, Tyler (2020) warns that the dynamic nature of the language can lead to runtime errors that are difficult to trace, particularly because it lacks a native type system. Managing dependencies in the NPM registry also remains a common challenge, often referred to as "dependency hell."

5.2.1 Use cases and best practices

JavaScript is the primary choice for creating interactive interfaces and single-page applications. Devographics (2026) explains that since the introduction of Node.js, it has also become a standard for full-stack development, allowing teams to use a single language for both front-end and back-end logic. In the context of university systems, Raymond (2023) observes its frequent use in modernizing legacy administrative tools by adding responsive layers that interact with existing databases via APIs.

To prevent maintainability issues, Padolsey (2020) suggests that developers should strictly adhere to clean code principles. This includes preferring `const` and `let` over the older `var` keyword and handling asynchronous code with `async` and `await` to make logic easier to debug. Using functional programming methods like `map`, `filter`, and `reduce` for array manipulation further leads to more predictable and declarative code.

5.2.2 Comparison to TypeScript and migration analysis

TypeScript functions as a statically typed superset of JavaScript, essentially adding a critical layer of type safety that the original language lacks. While JavaScript remains more flexible and is often faster for small-scale projects or rapid prototyping, Tyler (2020) points out that TypeScript provides significantly better IDE support, including more reliable autocomplete features and powerful refactoring tools. The primary trade-off in this transition is that TypeScript requires a compilation step to transform the code back into standard JavaScript. This process can occasionally slow down the development cycle in extremely large codebases if the build environment is not correctly optimized, though for most modern applications, the benefits of safety far outweigh this minor

include a highly scalable application where the logic remains readable and accessible for any future contributors at Mendel University.

5.4 Vite

Vite serves as the primary build tool and development server for modern web projects, representing a significant shift in how front-end applications are constructed. Unlike older bundling tools that attempt to process the entire application before the development server can even start, Padolsey (2020) explains that Vite leverages native browser ES modules to deliver source code on demand. This architectural approach eliminates the long wait times typically associated with large-scale projects, allowing the development server to start almost instantly regardless of how many components or files the application contains.

Beyond its rapid startup time, the tool dramatically improves developer productivity through its highly efficient Hot Module Replacement (HMR) system. Tyler (2020) notes that when a developer makes a change to a specific file, Vite only updates the affected module in the browser instead of reloading the entire page. This precision allows the application state to remain intact during development, which is particularly useful when working on complex forms. Furthermore, Vite handles TypeScript transpilation out of the box using Esbuild, which provides extremely fast build speeds compared to traditional compilers.

For the production deployment of the accreditation application, Vite provides a robust and stable foundation. It utilizes Rollup under the hood to optimize the final output through advanced techniques such as tree-shaking and code-splitting. These optimizations ensure that users at Mendel University only download the specific code necessary for the view they are currently accessing. By minimizing initial load times and reducing the overall bundle size, Vite significantly improves the responsiveness and perceived performance of the client-side application.

5.5 Scrumban

Agile development is characterized by an emphasis on iterative progress and constant flexibility, an approach that allows developers to respond effectively to changing requirements throughout the implementation phase. To manage the creation of the accreditation application, the team utilized Scrumban, a

hybrid methodology that combines the structured nature of Scrum with the fluid, continuous workflow of Kanban. As Aljehani et al. (2025) explain, this model removes the rigid constraints of fixed-length sprints while maintaining essential ceremonies like planning and reviews. This flexibility is particularly effective for projects where tasks vary significantly in complexity and priorities shift frequently.

The primary goal of adopting Scrumban is to optimize the overall flow of work through the use of a "pull-based" system. In this environment, tasks move from a backlog into active development only when the team has the necessary capacity to handle them. Aljehani et al. (2025) note that setting work-in-progress limits is a vital part of this process, as it prevents the team from becoming overwhelmed by too many simultaneous tasks. This focus on throughput ensures that specific functionalities, such as the publication section or the administration sub-applications, are completed and fully tested before the team begins work on new features.

The Smart MENDELU team is composed of students and academic staff who must manage project tasks alongside their primary studies and teaching duties. Because these participants do not work on the application full-time, individual availability often fluctuates. This environment creates an inherent risk where a single person's absence could potentially halt the development of the entire project. Scrumban mitigates this issue by making the current state of every task visible to the entire group, ensuring that progress is not dependent on the constant presence of a single individual.

5.5.1 Trello

Trello provides the visual framework necessary to implement these Scrumban principles effectively. The application utilizes a board-and-card system to represent the various stages of the project lifecycle (Atlassian, 2025). Each card contains specific details, technical requirements, and testing criteria for a single feature. These cards are organized into columns that represent stages like "Backlog," "Next Up," "Testing," and "Done." This high level of transparency allows for the immediate identification of bottlenecks in the development of the accreditation system.

Furthermore, Trello serves as a central knowledge hub where each card contains the necessary context for another team member to take over a task if the original owner becomes unavailable. As Parsons (2018) points out, this visibility prevents administrative modules from stalling and ensures that work continues without interruption. By maintaining a clear visual record of progress, the project remains resilient to the unique scheduling constraints often found in a university environment.

5.6 Git and Fork

Git and Fork served as the primary tools for version control and code collaboration during the development of the accreditation application. While the use of Fork was not strictly enforced as some developers preferred tools integrated directly into their IDEs, Git served as the underlying foundation for all code management.

Specifically, the university's GitLab instance acts as the primary version control system, tracking every incremental change made to the source code. GitLab (2026) describes how this system facilitates seamless collaboration between team members, allowing developers to work on separate features in isolated branches without interfering with the main codebase. This branching strategy ensures that new code is only merged into the stable version of the software after it has been thoroughly verified.

To simplify these version control operations, many developers utilized Fork, a desktop Git client. According to Pristupov (2025), Fork provides a visual representation of Git repositories that simplifies complex tasks like rebasing and resolving merge conflicts through an intuitive graphical interface. For a team with varying levels of experience, this tool reduces the risk of accidental data loss in the version history. Furthermore, the visual commit timeline makes it easier for the team to track the historical progress of specific tasks over the course of the project.

5.7 SonarQube and Swagger

SonarQube and Swagger were the primary tools used for quality assurance and documentation.

SonarQube analyzes the source code to identify potential security risks and technical debt. It automatically scans the repository for bugs and "code smells" that could lead to future maintenance issues. By establishing automated quality gates, the tool ensures that the TypeScript implementation remains clean and secure. This process is essential for long-term projects like university systems where multiple developers will interact with the code over several years (SonarSource Sàrl, 2026).

Swagger facilitates communication between the front-end and back-end development stages. It provides interactive API documentation that describes every available endpoint and the required data structures. This tool allows the Vue.js application to be developed in parallel with the server-side logic.

Developers can test responses and verify the integration of new forms without needing to manually inspect the backend source code (SmartBear Software, 2026).

5.8 Docker

Docker containerizes the application to ensure consistency across different technical environments. It bundles the software with its specific libraries and configurations into a single image (Docker Inc., 2026). This isolation prevents conflicts between different projects or versions of Node.js on a single machine and makes the deployment to university servers more reliable by providing a predictable execution environment that matches the developer's local setup.

6 Design specifications

The design of the Accreditation application was not developed in isolation but as a core component of the broader Smart MENDELU ecosystem. The visual language, user experience patterns, and interface logic follow the comprehensive design system and high-fidelity wireframes established by Veverka (2025). This chapter focuses on the technical analysis and adaptation of these designs into functional frontend specifications. By aligning with these predefined blueprints, the implementation ensures visual consistency across the university's digital tools while allowing the development to focus on the technical challenges of complex form logic and real-time data synchronization.

6.1 User interface and experience architecture

The architecture of the user interface is built upon a modular and hierarchical structure designed to handle the dense information requirements of academic accreditation. Following the project's design standards, the application utilizes a "dashboard-first" approach where the primary navigation is situated in a persistent sidebar. This allows users to switch seamlessly between their personal profile, the central accreditation forms, and administrative sub-modules without losing their place in the workflow.

Key architectural features of the UI include:

- **Unified visual identity:** The application strictly adheres to the Mendel University design manual, utilizing a specific color palette (primarily MENDELU green and neutral grays) and typography that mirrors the university's official branding to foster user trust and familiarity.
- **Layout consistency:** A standardized grid system is employed to organize form fields, ensuring that whether a user is filling out a faculty-level form or a program-specific form, the spatial distribution of inputs, labels, and help text remains predictable.
- **Information scaffolding:** To prevent administrative burnout, the interface uses "progressive disclosure" techniques. Complex forms are broken down into logical sections or tabs, preventing the user from being overwhelmed by too many inputs at once, a direct response to the "burnout" issues identified in the previous manual spreadsheet system by Veverka (2025).

- **Navigation and breadcrumbs:** Given the depth of the accreditation dossiers, a robust breadcrumb system and clear header signposts are implemented to ensure users always understand their current location within the multi-level accreditation hierarchy.

6.2 Component-based design approach

The technical implementation of the accreditation system follows a modular architecture that mirrors the visual logic found in the project's Figma specifications. By breaking down the complex accreditation forms into smaller, manageable units, the application achieves a high degree of maintainability and code reuse. This approach ensures that updates to the university's visual identity can be applied globally without necessitating a full rewrite of the business logic.

To translate the high-fidelity wireframes provided by Veverka (2025) into Vue.js, the project adopted the principles of "Atomic Design" (Ramezani et al., 2025). This methodology categorizes UI elements into hierarchical levels:

- **Atoms:** The most basic building blocks, such as standardized buttons, input fields, and typography styles, which were mapped directly to Vuetify's base components and styled via a shared library.
- **Molecules:** Combinations of atoms that form functional groups, such as a labeled input field with validation messaging or a date picker specifically configured for accreditation deadlines.
- **Organisms:** Complex sections of the interface, such as the entire "Publications" management block or the "A-I Form Header," which combine multiple molecules into a distinct part of the user journey.
- **Templates and pages:** The final assembly of these organisms into the specific views defined in Figma, such as the guarantor-specific view, the read-only view or the administrator dashboard.

Consistency across the Smart MENDELU ecosystem is maintained through a shared frontend library. While Vuetify serves as the underlying structural framework, the application utilizes a centralized style library within the [shared-frontend](#) repository. This repository contains the source of truth for the university's design tokens, including precise hex codes for the "MENDELU Green," standardized spacing units, and font-weight definitions. In practice, this means that the implementation of the B-I form does not define its own colors. Instead, it references shared variables, ensuring that if the university's unified design system is updated in the future, as is currently being piloted with the Attendance application, the accreditation system can be updated with

minimal friction.

Although the accreditation process involves dense data entry traditionally performed on desktop workstations, the implementation follows a responsive design strategy to ensure accessibility across various devices. The layout logic utilizes a fluid grid system provided by Vuetify, which allows form sections to stack vertically on smaller screens while maintaining a multi-column view on high-resolution monitors. To preserve the integrity of the original desktop-first designs, specific breakpoints were defined to handle complex elements like the accreditation status tables. In these cases, the interface utilizes horizontal scrolling or simplified "card views" for mobile users, ensuring that the critical administrative data remains readable and the "save" and "submit" actions remain easily accessible regardless of the user's hardware.

6.3 State and data flow design

The transition from static Figma designs to a dynamic application requires a robust strategy for managing data flow and the reactive state of the user interface. Because the accreditation process involves a complex hierarchy of users ranging from faculty representatives to external teachers, the interface must adapt its behavior and visibility based on the current context and user permissions.

A key requirement identified in the Figma specifications is the need for distinct views for different user roles. Rather than creating entirely separate pages for each role, the implementation utilizes a centralized "Guard" pattern. Through the use of Vue's conditional rendering directives (`v-if` and `v-else`), the application dynamically toggles the visibility of action elements, such as "Edit," "Submit," or "Approve" buttons. For instance, a course guarantor is presented with an editable view of the B-I form, while a member of the evaluation council accessing the same URL is served a high-fidelity "read-only" version of the interface. This approach ensures that the business logic remains centralized while the UI adapts to the user's specific authorization level.

Managing the state of the A-I and B-I forms presents a significant design challenge due to the large number of fields and the need for persistent data. The implementation employs a reactive state management pattern to track the "dirty" state of a form, identifying when a user has modified a field but not yet saved it. This state is visually communicated to the user through status indicators and warning prompts if they attempt to navigate away from an unsaved form. Furthermore, the design of the data flow includes an "immediate feedback" loop, where validation logic checks the input against university standards in real-time, highlighting errors as they occur rather than waiting for a final submission.

6.4 Security and access control design

The security architecture of the client-side application is designed to be invisible to the user while maintaining strict institutional standards. Security is not merely a backend concern but is integrated into the design through several frontend mechanisms:

- **Authentication integration:** The design of the login flow is integrated with the university's central authentication services. On the frontend, this involves managing secure session tokens and ensuring that the application state is cleared immediately upon logout to prevent unauthorized access on shared university workstations.
- **Granular component shielding:** Beyond simple page-level access, the design includes "component-level shielding." Using specialized wrapper components, the implementation can hide or disable specific sensitive data fields within a form based on the user's specific role, even if they have access to the page itself.
- **Read-only enforcement:** To preserve the integrity of the accreditation records, the application implements a global "Read-only" mode. This state is triggered when a form has already been submitted or is under review, effectively locking the UI components and preventing any further modifications.

7 Implementation

The implementation phase represents the transition from conceptual design and technical requirements to the creation of a functional, high-performance software tool. Following the design specifications and technical architecture analyzed in previous chapters, this phase focuses on the practical development of the client-side components of the accreditation system. The primary objective was to translate the high-fidelity Figma wireframes into a resilient Vue.js application that integrates seamlessly into the Smart MENDELU ecosystem, providing a stable and intuitive environment for academic staff to manage complex administrative dossiers.

This chapter details the specific technical strategies and coding practices used to realize the application's core modules, including the A-I and B-I forms and the research publications section. It describes the project's internal directory structure, the utilization of the Vuetify framework alongside the shared style library, and the methods employed to synchronize the application state with backend services. By emphasizing modularity and code reusability, the implementation not only fulfills the immediate functional needs of the university but also establishes a scalable foundation for future developments within the institution's ongoing digitalization initiative.

7.1 Publications in User profile

The Publications tab in the user profile acts as a centralized dashboard to track and display academic output. It manages a wide range of data including publication lists, research identifiers, and various citation metrics. The interface adapts its functionality based on the origin of the user data. Internal users have their records synchronized from the university information system, while external users require manual data management.

To maintain a clean codebase, the view consists of several modular sub-components. The `PublicationsTable` handles the display of existing research outputs. Data entry for research identifiers like ORCID and Scopus Author ID occurs within the `PublicationIdentifiers` component. Quantitative data such as citation counts and H-index values are managed through dedicated forms. This separation of concerns ensures that each part of the interface remains easy to test and maintain.

Table 7.1 Publication components.

Component	Responsibility
<code>PublicationsTable</code>	Displays a list of existing publications and research outputs.
<code>PublicationIdentifiers</code>	Form for managing ResearcherID, ORCID, and Scopus Author ID (Editable for external users).
<code>PublicationCitation</code>	Form for inputting citation counts across WOS, Scopus, and other platforms.
<code>HIndex</code>	Form for managing H-Index values from Web of Science and Scopus.

7.1.1 Data model and user logic

The implementation uses a complex `formData` object to synchronize state between the user interface and the backend. This object tracks specific academic metrics such as citation counts across Web of Science and Scopus. It also includes fields for the number of supervised bachelor, diploma, and doctoral theses. Maintaining this centralized object allows for consistent data validation across the entire profile section.

Logic within the component distinguishes between user types by checking specific route parameters. If a university identification ID is absent, the system treats the profile as an external user. In this mode, research identifiers become fully editable. The application uses specific functions to fetch and save external data. Conversely, internal user profiles display identifiers as read-only links. These records are pulled directly from the university store and cannot be modified within the accreditation application.

Academic metrics

- Citations: `citationWOS`, `citationScopus`, `citationOther`
- Publication Counts: `publicationNumberWOS`, `publicationNumberScopus`, `publicationNumberOther`
- H-Indices: `hIndexWOS`, `hIndexScopus`

Figure 7.1
Publications section in User profile (Veverka, 2026)

Supervision statistics

- [supervisedBachelorThesesNumber](#)
- [supervisedDiplomaThesesNumber](#)
- [supervisedRigorousThesesNumber](#)
- [supervisedDissertationsNumber](#)

Research identifiers

- [researcherId](#) (linked to ResearcherID/Publons)
- [orcid](#) (linked to ORCID)
- [scopusAuthorId](#) (linked to Scopus)

7.1.2 Technical implementation

State management relies on the [useUserProfileStore](#) to coordinate global data. Using the [storeToRefs](#) utility ensures that the component remains reactive to changes in the store without unnecessary re-renders. A watcher monitors the component mounting process and loading states to trigger data fetching at the appropriate time. This reactive pattern keeps the user interface synchronized with the underlying data layer.

The data fetching workflow follows a three-step process. First, the application identifies the user type from the URL. It then executes the necessary API calls to retrieve academic activities or external personal information. Finally, a mapping function transfers the retrieved data into a local reactive reference for safe editing. This prevents direct modification of the global store until the user explicitly saves their changes.

The saving mechanism employs a modular function that supports partial updates. By passing a specific string parameter to the save function, the system determines which API endpoint to target. This parameter also controls which loading indicators appear and which localized messages the user sees upon success or failure. This approach reduces code duplication and provides a consistent experience across the different forms in the publication section.

7.2 Form A-I

Form A-I is the main administrative record used to document the foundational details of an accreditation file. It acts as a central repository for metadata

regarding the university and the specific study program undergoing review. The form tracks various identifiers including internal approval dates and ISCED-F classifications. It also links the application to external institutional partnerships and official university regulations.

7.2.1 Core data fields

The form manages a mix of static and interactive data fields. Institutional metadata such as faculty names and approving authorities remain read-only to ensure consistency with the university information system. In contrast, administrative details require manual input to specify the type of accreditation and the name of any cooperating institutions. Users must also provide a code and a written justification for the field of education classification.

A significant portion of the form handles external documentation and resource links. This includes the electronic version of the application and various study support materials. The system automatically lists relevant attachments based on previously uploaded files such as training agreements. By consolidating these links and files, the form provides a complete overview of the administrative requirements for the program.

7.2.2 Architecture

The technical structure of the form follows a modular design that separates the visual interface from the underlying logic and data management.

The user interface is built in `FormAI.vue` view using the Vuetify framework and functions as the primary interaction layer. It dynamically switches between editable and read-only modes depending on the current status of the application. This ensures that a form already submitted for approval cannot be modified by mistake. To assist the user, the view includes a panel for field-specific instructions and a dialogue that warns against losing unsaved changes. Header buttons coordinate major actions like sending the form for rework or final approval.

The business logic is contained within a dedicated composable `useAI-Form.js`. This module enforces specific validation rules such as verifying that all links use the secure protocol. It also manages the transformation of raw data from the backend into reactive objects for the front end. When preparing a submission, the logic ensures that the data structure matches the requirements of the API. It also generates localized lists for dropdown menus to facilitate the selection of classification codes and accreditation types.

M

Akreditační spisy

Aktuální role: Zástupce fakulty
Přepnout
xuzivatel

Moje spisy > Otevřená informatika > Formulář A-I

Formulář A-I

Další akce

Komunikace

Uložit změny

Stav formuláře: Vyžaduje moji akci

Název vysoké školy	Mendelova univerzita v Brně
Název součásti vysoké školy	Provozně ekonomická fakulta
Název studijního programu	Otevřená informatika
Typ žádosti o akreditaci	Žádost o udělení akreditace studijnímu programu
Schvalující orgán	Rada pro vnitřní hodnocení Mendelovy univerzity v Brně
Odkaz na vnitřní předpisy univerzity	http://mendelu.cz/24997-vnitri-predpisy-univerzity
Odkaz na vnitřní předpisy fakulty	http://www.pef.mendelu.cz/uredni-deska-dalsi-predpisy
Odkaz na elektronickou podobu žádosti	Pro otevření elektronické verze klikněte zde
Odkaz na příklady smluv o zajištění odborné praxe	-
Odkaz na poslední zprávu o vnitřním hodnocení vysoké školy	-

Název spolupracující instituce dle §81 nebo §95 odst. 4 ZVŠ

Datum schválení žádosti

Odkaz na studijní opory pro kombinovanou / distanční formu studia

ISCED F

Stručné zdůvodnění ISCED F

Seznam příloh

- Smlouvy o zajištění odborné praxe
 - Název první smlouvy
 - Název druhé smlouvy
- Název první přílohy
- Název druhé přílohy
- Název třetí přílohy

Figure 7.2
Form A-I (Veverka, 2026)

A Pinia store `FormAStore.js` serves as the central source of truth for all data related to Form A-I. It handles the persistence of information by executing requests to the server. The store is responsible for determining whether the current user has permission to edit the form based on its lifecycle stage. Additionally, it contains fallback mechanisms that provide default data structures if the database schema does not match the expected API response. This prevents the application from failing if there are discrepancies between the frontend and the backend versions.

7.2.3 Key dependencies and connections

Form A-I does not operate in isolation. It relies on a network of specialized Pinia stores to populate its interface and verify data consistency. The `FilesStore` manages metadata for all uploaded documents and confirms whether the required electronic version of the application is present. For classification data, the `TypesStore` provides the global list of ISCED-F codes and internal university regulations. Administrative settings like application types and list of approving authorities are retrieved through dedicated stores. Localized institutional names are provided by the `UniversitiesStore`. The application also interacts with the `ProfessionalPracticeWorkplacesStore` to check for existing workplace agreements and display relevant notifications.

Logic modules called composables handle specific behaviors and safeguards within the form. The `useFormApproval` module manages the complex transitions between workflow states such as moving a document from a rework phase to final approval. To keep track of where the component sits in its lifecycle, the system uses `useFormStatus`. Data integrity is further protected by the `useUnsavedChangesGuard`. This utility monitors the local state for any modifications and triggers a warning if the user attempts to leave the page before saving their progress.

Several shared components and views provide structural consistency. The `TitleRowButtons` component serves as a standard header that houses primary action buttons. If a user needs to discuss specific queries or rework requirements, the interface links directly to the `forms-communication` view. During the data entry process, the `InputDescriptionPanel` functions as a contextual sidebar. This panel updates dynamically to show help text related to whichever input field the user is currently editing.

7.2.4 Workflow Lifecycle

The operation of the form follows a structured sequence that begins as soon as the component mounts. The application first gathers all necessary file metadata and form records from the backend stores. Once the data is loaded, a permission check occurs. The store evaluates the current status of the form to decide if the user, such as a program guarantor, has the authority to edit the fields or if the interface should remain read-only.

User interaction triggers real-time logic and validation. As the user fills in the various sections, the `useAIForm` composable checks inputs for errors or missing information. The system also performs automatic formatting to ensure data quality. For example, dates are converted to the correct local format and web addresses are automatically prefixed with the necessary security protocol. These background tasks reduce the manual effort required from the user and prevent common data entry mistakes.

The lifecycle concludes with the submission process. Users have the option to save a draft at any time, which updates the database without advancing the workflow. When the user is ready to finalize the document, the system executes a full validation check. If the form meets all requirements, an approval popup appears. This action moves the form to the next stage of the accreditation process and usually locks the data from further editing.

7.3 Form B-I

Form B-I serves as the primary data entry point for the basic information of a study program. Its main purpose is to consolidate both qualitative descriptions and quantitative regulatory data into a single administrative record. The qualitative sections focus on narrative elements such as study goals, graduate profiles, and professional applicability. Simultaneously, the form tracks critical quantitative metrics including the standard study duration, education field percentages, and specific details regarding regulated professions. By combining these different data types, the form provides a complete overview of the program's structure and objectives.

7.3.1 Core architecture

The implementation of the form follows a standard Vue 3 Composition API pattern. This approach ensures a clear separation of concerns by dividing the application logic into distinct layers. The `FormBI.vue` file acts as the view layer and is responsible for managing the user interface, template structure, and UI-specific states like loading indicators or dialog windows. It orchestrates the interaction between the user and the underlying system.

The business logic is encapsulated within the `useBIForm.js` composable. This module handles form validation rules and maintains a reactive data object used specifically for editing. By isolating these rules from the view, the logic remains easier to test and modify. The `FormBIStore.js` Pinia store manages the persistence layer and synchronization with the backend API. It also evaluates role-based access control to ensure that only authorized users can modify specific sections of the data.

7.3.2 Functional sections of the form

The user interface is logically organized into three distinct sections, the read-only metadata header, the fillable and editable fields and the workflow actions, that change behavior based on the current state of the application.

The top of the form contains a header that displays immutable information fetched from the university information system or the parent study program. This section provides the foundational context for the application and includes the university name, study program title, and the designated guarantor. It also lists the program type, profile, and the language of instruction. These fields are read-only to prevent unauthorized changes to data that is managed at a higher institutional level.

When the form is placed in edit mode, several interactive sections become available for data entry. Users can specify the standard study time using a numeric input and select the appropriate academic degree from a dropdown menu. The form also includes specialized toggles for regulated professions and security experts. If these options are activated, a conditional selector appears to allow the user to choose the correct recognition authority.

Managing education fields requires a more complex interaction. The system uses a dynamic component called `AddTwoValuesAutocom` to allow users to build a list of academic fields and assign a percentage share to each. For qualitative information, the form provides large text areas. These sections are used to document the study plan rules, graduate profiles, and acceptance

Akreditační spisy Aktuální role: Zástupce fakulty Přihlásit Kuchyně

Moje spisy > Otevřená informatika > Formulář B-I Další akce Komentáře Místi zprávy

Formulář B-I

Stav formuláře: **Vytváříte nový**

Název studijního programu	Otevřená informatika
Typ studijního programu	Magisterský
Profil studijního programu	Akademicky zaměřený
Forma studia	Prezenční
Jazyk studia	Český jazyk
Garant studijního programu	Bc. Adam Veverka
Rigorózní řízení	Ne
Udělovaný akademický titul pro rigorózní řízení	-

Standardní doba studia

Udělovaný akademický titul

Zaměření na přípravu k výkonu regulovaného povolání

☒ Ano

☐ Ne

Zaměření na přípravu odborníků z oblasti bezpečnosti České republiky

☒ Ano

☐ Ne

Uznávací orgán

Oblasti() vzdělávání a u kombinovaného studijního programu podíl jednotlivých oblastí vzdělávání v %

Oblast vzdělání	Podíl oblasti v %
Oblast vzdělání	Podíl oblasti v %

+ Přidat oblast

Cíle studia ve studijním programu

Profil absolventa studijního programu

Předpokládaná uplatnitelnost absolventů na trhu práce

Pravidla a podmínky pro tvorbu studijních plánů

Podmínky k přijetí ke studiu

Předpokládaný počet uchazečů zapsaných ke studiu ve studijním programu

Návaznost na další typy studijních programů

Figure 7.3
Form B-I (Veverka, 2026)

conditions. This structured layout ensures that all regulatory requirements are met while allowing for detailed descriptions.

The lifecycle of the form is controlled through a series of workflow actions integrated into the standard header. These actions are triggered by the TitleRow-Buttons and managed through various approval popups. The save function allows developers or academic staff to persist their current progress without advancing the form to the next stage. Once the information is complete, the submit action initiates the formal approval process. Users with managerial roles can then use the approve or return buttons to either progress the application or send it back for necessary reworks.

7.4 Form status

The functionality and accessibility of each form within the accreditation application are determined by its current status, which is dynamically calculated based on the specific role of the user. This system ensures that every person interacting with the file has the correct level of authority and access during each stage of the administrative workflow. Because permissions are not static, the application evaluates these variables every time a component mounts or a user navigates between different sections of the program. This dependency between the workflow stage and user permissions prevents unauthorized modifications while guiding the user through necessary actions.

Five primary roles define the hierarchy within the application and dictate the level of control a user has over a document. The Faculty Representative holds the highest authority, followed by the Program Guarantor and the Course Guarantor. The Teacher and Form Owner represent the lower tiers of this structure. If a single user is assigned multiple roles for the same file, the system uses a prioritization function to resolve them to the most significant role. This hierarchy is essential because it determines who can advance the document through the approval process and who is permitted to view or modify sensitive qualitative data.

From the perspective of the user, the form exists in one of several states that indicate both its progress and any required interaction. The "Action required" status signifies that the user must edit or approve the document to move it forward. Conversely, statuses such as "Waiting" or "Sent for approval" indicate that the form is currently being reviewed by another participant in the hierarchy. Once a form passes all administrative checks and is finalized, it moves to the "Validated" status. In cases where a user has no involvement with a specific form or the process has moved beyond their involvement, the status transitions to "No access" or "Read only".

Akreditační spisy Aktuální role: Zástupce fakulty Přepnout xuzivatel

Moje spisy > Otevřená informatika

Otevřená informatika Zobrazit elektronickou verzi Zamknout Exportovat do PDF

Formulář A-I Vyžaduje moji akci Vstoupit

Formulář B-I Schválený Vstoupit

Formuláře B-IIa Vračen na přepracování Vstoupit

Formuláře B-IIb Vyžaduje moji akci Vstoupit

Formuláře B-III Vyžaduje moji akci Vstoupit

Formulář B-IV Vyžaduje moji akci Vstoupit

Formuláře C-I Vyžaduje moji akci Vstoupit

Formulář C-II Čeka se na cizí akci Vstoupit

Formulář C-III Pouze ke čtení Vstoupit

Formulář C-IV Čeka se na cizí akci Vstoupit

Formulář C-V Pouze ke čtení Vstoupit

Formulář D-I Čeka se na cizí akci Vstoupit

Formulář E Čeka se na cizí akci Vstoupit

Přílohy Čeka se na cizí akci Vstoupit

Přehled spisu Upravit

- Aktuální stav spisu: **Vyžaduje moji akci**
- Součást VŠ: **Provozně ekonomická fakulta**
- Studijní program: **Otevřená informatika**
- Typ studijního programu: **Magisterský**
- Profil studijního programu: **Akademicky zaměřený**
- Forma studijního programu: **Prezenční**
- Výukový jazyk: **Český jazyk**
- Specializace: **Datová analýza a strojové učení, Uživatelská rozhraní, Zahraniční mobilita**
- Garant: **Ing. Alex Addams Gordon Ph.D MSc**

Figure 7.4
Example file with forms with different statuses (Veverka, 2026)

Table 7.2 Ability of a person with a certain role to modify the form based on its status.

	Action required	Sent for approval	Waiting	Validated	Read only	No access
Faculty representative	Yes	-	Yes	No	No	No
Guarantor of program	Yes	No	Yes	No	No	No
Guarantor of course	Yes	No	No	No	No	No
Teacher	Yes	No	No	No	No	No

The combination of the user role and the form state directly dictates the behavior of the user interface. When the system identifies a "No access" status, it blocks the content entirely and displays a specific screen to inform the user they lack permissions. In the "Read only" state, the document content remains visible for informational purposes, but the logic layer disables all input fields to protect the integrity of the information. Full modification is only permitted when the status explicitly matches an active state for that specific role. An exception is made for high-level roles like Faculty Representatives, who may retain editing rights during certain waiting periods to facilitate quick corrections without restarting the entire approval chain.

7.4.1 Form status backend implementation

The backend manages each form through a structured data model that ensures persistence and traceability. A status field indicates the position of the document in the accreditation process using standardized numeric identifiers known as `FormApprovalStatusIds`. These identifiers represent specific states: 1 for approved, 2 for rejected, and 3 for submitted. This numeric mapping allows the server and the client application to communicate the progress of a document consistently. The system also records timestamps for creation, submission, and finalization, which helps the university assess workflow efficiency and detect delays.

Accountability is handled through an audit trail that records every status change instead of overwriting existing data. Each entry captures the previous and new states, the timestamp, and the identity of the responsible user. This creates a complete and reliable history of modifications for administrative oversight and ensures that any change can be traced back to its origin.

A finite state machine governs how forms move between states by enforcing

Table 7.3 Possible actions based on user role and form status.

	Action required	Sent for approval	Waiting	Validated	Read only	No access
Faculty representative	Save changes, Communication, Approve, Send for rework	-	Save changes, Communication, Approve	Communication, Mark as incomplete	Communication	-
Guarantor of program	Save changes, Communication, Send for approval, Send for rework	Communication	Save changes, Communication, Send for approval	Communication	Communication	-
Guarantor of course	Save changes, Communication, Send for approval	Communication	-	Communication	Communication	-
Teacher	Save changes, Communication, Send for approval	Communication	-	Communication	Communication	-

defined transitions. This logic ensures that documents follow the required sequence without skipping essential steps. As a result, no form can reach a validated state without passing all prior stages. Before any transition occurs, the backend validates the form by checking that required fields are complete and meet technical standards. If an issue is found, the system blocks the change and returns a clear error message to the user.

Status updates also trigger automated actions to keep the process moving without manual follow-up. For example, when a form moves to the submitted state, the system automatically notifies the relevant faculty members that a review is required. These background tasks reduce the administrative burden on participants and help maintain project momentum even when individual availability fluctuates.

The API enforces authorization by checking each user's role against the

permissions required for a given action. This ensures that only authorized users, such as program guarantors or faculty representatives, can perform critical updates. To maintain consistency, all changes to the form and its corresponding audit log are executed within a single database transaction. This atomic approach ensures that either both updates succeed or neither does, preventing the data from becoming desynchronized.

Once the transaction completes, the API returns the updated status ID to the client. This allows the frontend to refresh immediately and reflect the correct visual state and user permissions. By providing the updated `FormApprovalStatusIds` in the response, the system ensures the user interface stays synchronized with the latest version of the data stored on the server.

7.4.2 Form status implementation on front-end

Table 7.4 Possible actions based on user role and form status.

	Guar- ProgAction- Required	SentForApproval	ReturnedForRework	Validated	FileLocked
Faculty rep- representative	Waiting	Action required	Waiting	Approved	Read only
Guarantor of program	Action required	Waiting	Action required	Approved	Read only
Guarantor of course	No access	No access	No access	No access	No access
Teacher	No access	No access	No access	No access	No access

The front-end implementation translates raw data from the server into a functional and secure user interface. While the backend manages the persistence of statuses, the client application is responsible for interpreting those states to define how a user can interact with each form. This process involves a combination of data synchronization, complex mapping logic, and visual feedback mechanisms.

The client application begins the synchronization process by executing GET requests to fetch form-specific data and the current backend status IDs. To identify the permissions of the active user, the system integrates with the `UserRoleStore`. This store provides a list of all academic and administrative roles assigned to the user for the specific accreditation file. Once retrieved, this status and role information is saved into a `Pinia` store. This centralized storage ensures that the data is accessible across all components throughout the

user session without requiring redundant API calls.

At the heart of the front-end implementation is a logic engine that determines the final interface state for a specific user. Because a single individual may hold multiple positions, the system resolves the hierarchy by prioritizing the most significant role. A Faculty representative role takes precedence over a Teacher role to ensure the highest level of authority is applied. The final UI status is then calculated by crossing the backend status ID with this resolved role through a predefined matrix.

The application also uses specialized "table groups" to handle different administrative requirements based on the form type. These groups apply distinct mapping rules depending on the specific document being viewed. For instance, the logic for Form B-I might allow a teacher read-only access, whereas other forms may block access for that role entirely. This approach allows the system to remain flexible while maintaining strict control over the accreditation workflow.

Table 7.5 Possible actions based on user role and form status.

	Guar- ProgAction- Required	SentForApproval	ReturnedForRework	Validated	FileLocked
Faculty rep- representative	Waiting	Action required	Waiting	Approved	Read only
Guarantor of program	Action required	Waiting	Action required	Approved	Read only
Guarantor of course	Read only	Read only	Read only	Read only	Read only
Teacher	Read only	Read only	Read only	Read only	Read only

The application enforces user permissions primarily through the `getIs-ReadOnly` function. This utility automatically enables or disables all input fields based on the calculated UI status. However, the system includes contextual overrides for specific high-level roles. Faculty Representatives are permitted to maintain editing rights even during "waiting" states. This allows them to make quick administrative corrections without restarting the entire approval chain.

Interactivity is also tied directly to the saving mechanism. If the status logic determines that a user should not have permission to modify a form, the interface blocks unauthorized saving attempts at the component level. This prevents any data from being sent to the server if the user is in a read-only or restricted state. By connecting the status logic to the action buttons, the application ensures that the user interface remains synchronized with the backend security policies.

Technical states are translated into user-friendly labels to guide the user

through the process. Labels such as "Action required" or "Sent" provide clear instructions on the current stage of the work. To provide immediate visual feedback, the application uses a standardized color palette. Red indicates that an action is necessary, orange signifies a waiting period or that a form has been sent, and green marks a validated document. Grey is reserved for read-only access where no interaction is possible.

Security and usability are further enhanced by integrating status checks with Vue Router guards. These safeguards prevent users from navigating to forms where they have no designated role or permission. By verifying the status and role before a page even loads, the application provides a smooth experience that keeps users within the boundaries of their specific responsibilities.

7.5 Form approval process

The approval process represents the dynamic progression of a document through the institutional hierarchy. While the status system defines general permissions and visibility, the approval workflow manages the actual movement of data between different levels of authority. This mechanism ensures that every change is reviewed by the appropriate stakeholders and that the transition from a draft to a validated document follows a strict administrative sequence. By formalizing these transitions, the application prevents procedural errors and maintains the integrity of the accreditation records.

7.5.1 Transition-based authority

The ability to advance a form through the workflow is governed by a specific layer of logic that identifies the active role of a user. Unlike general viewing permissions, actionable role resolution focuses on whether a user has the authority to change the current state of a document. The system utilizes `useUserRoleStore.js` to evaluate all roles assigned to a user for a particular file. If a user holds multiple positions, the application identifies the highest-priority role to determine which workflow actions are available. This distinction is critical because it ensures that a Faculty Representative can always act as a final approver even if they are also listed as a teacher or a guarantor on the same file.

The technical implementation of this authority is centered in the `TitleRowButtons.vue` component. This component serves as the primary interface

for workflow transitions and relies on a configuration known as the `roleActionMap`. This mapping functions as a matrix that dictates the visibility of the "Approve," "Return," and "Send" buttons based on a combination of the user's active role and the current backend status. For example, the logic may hide the "Send" button for a Guarantor once the form has already been submitted, while simultaneously revealing the "Approve" and "Return" buttons for the Faculty Representative.

By hardcoding these relationships into the `roleActionMap`, the application ensures that users are only presented with valid choices. This prevents accidental submissions or unauthorized approvals at the interface level. The component continuously listens for changes in the form state, allowing the available actions to update immediately as the document moves through the lifecycle. This granular control over button visibility simplifies the user experience by removing irrelevant options and clearly highlighting the next steps required to progress the accreditation file.

7.5.2 The lifecycle of an approval workflow

The progression of a form through the accreditation system is a multi-stage journey that balances data entry with formal administrative oversight. Each document begins as a draft and undergoes a series of transitions that lock or unlock specific capabilities based on the current phase of the lifecycle. This structured movement ensures that data remains consistent and that all modifications are accounted for before the form reaches its final validated state.

Before a form enters the formal approval chain, users interact with it through a phase of data entry and preparation. The "Save Changes" mechanism is the primary tool during this stage, allowing users to persist their work without altering the workflow state. This action triggers the `updateMainFormData` function, which synchronizes the current state of the Vue.js reactive components with the backend database. Because this operation does not involve a status transition, the form remains in the "Action required" or "Waiting" state. This allows for long-term drafting sessions where information can be refined over multiple days without prematurely triggering a review by higher-level authorities.

The submission phase represents the first major transition in the workflow. Once the responsible user, such as a teacher or program guarantor, is satisfied with the document, they trigger the "Send" action. This operation executes an API call with a `formApprovalStatusId` of 3. Submitting the form effectively transfers responsibility to the next level of the hierarchy, typically moving the document into a pending state. For the sender, the form usually becomes read-only at this point to prevent further edits while the review is underway. This transition signals to the system that the content is ready for formal evaluation

by a faculty representative or other designated approvers.

Approvers evaluate the submitted data and have two primary options to progress the workflow. If the document meets all institutional and regulatory requirements, the approver selects the "Approve" action. This sends a `form-ApprovalStatusId` of 1 to the backend, which transitions the form to the "Validated" status. This is a terminal state for standard workflows where the data is locked and marked as officially completed.

If the approver identifies errors or requires additional information, they utilize the "Return for Rework" action. This choice assigns a `formApprovalStatusId` of 2 and requires the approver to provide a specific reason for the rejection. The document then reverts to the previous owner, and the status changes back to "Action required." This loop ensures that all corrections are documented and that the form only proceeds when all quality standards are met.

Even after a form has reached the validated state, the system provides a mechanism for late-stage corrections through workflow reversion. A faculty representative can use the "Mark as In-Progress" function to reopen a finalized document. This action sends a specific payload that resets the form status, allowing for necessary edits that may have been overlooked during the initial review. Reverting a form is a deliberate action that restarts the approval chain, ensuring that even late-stage adjustments are subject to the same rigorous validation and oversight as the original submission. This functionality provides a critical safety net for handling complex accreditation files where institutional requirements may shift during the process.

7.5.3 Technical architecture of the approval engine

The structural integrity of the approval workflow relies on a centralized logic layer that separates the user interface from the backend communication. By consolidating these operations, the application maintains a consistent behavior across various form types while reducing code duplication. This architecture ensures that any change to the administrative rules or API structures only needs to be implemented once to take effect throughout the entire system.

The orchestration of all approval-related tasks is handled by the `useFormApproval.js` composable. This module acts as a shared service that abstracts the complexity of API calls away from the individual form components. Rather than each view containing its own logic for submitting or rejecting data, they all consume this centralized composable. This design pattern promotes a unified approach to error handling and request management. When a component triggers an action like "Approve" or "Send," it simply passes the necessary parameters to the composable, which then manages the lifecycle of the asynchronous request.

A key feature of the approval engine is its ability to resolve API endpoints

dynamically. Because the application manages a diverse range of forms, such as A-I or C-II, the backend routes vary depending on the document type. The composable handles this by internally constructing the appropriate URL based on the form identifier provided during the function call. For instance, a request for Form A-I is automatically directed to `/forms-a/i/approvals`. This logic allows the frontend to scale easily as new forms are added to the system without requiring manual configuration for every new endpoint. The dynamic resolution ensures that the correct business logic is triggered on the server while keeping the frontend code clean and maintainable.

Maintaining a synchronized state between the client and the server is essential for a smooth user experience. After a successful approval action, the application must immediately reflect the updated status to prevent the user from interacting with stale data. The system achieves this by calling the `getSiteInformation` function immediately after the API response is received.

This call forces the UI to re-evaluate the user's role and the new form status. The results of this synchronization update the Pinia store, which in turn refreshes the permissions across the application. If a form moves from "Action required" to "Sent for approval," the synchronization ensures that the input fields are locked and the action buttons disappear instantly. This reactive cycle is the final step in every approval transaction, providing the user with immediate confirmation that their action was recorded and that the workflow has progressed.

7.5.4 Communication and feedback loops

The success of the accreditation process depends on clear dialogue between authors and approvers. While the status system manages the technical state of a document, the communication module provides the context behind every administrative decision. This module acts as a bridge between participants, ensuring that rejections or requests for changes are accompanied by specific instructions. By integrating these discussions directly into the application, the system eliminates the need for external email chains and keeps all relevant information within the document context.

The communication interface is designed as a chat-like timeline that records the history of a specific form. Whenever an approver returns a form for rework, the system opens a dialog requiring a text description of the issues found. This message is then saved alongside the status transition. The resulting timeline presents a chronological view of the workflow, showing exactly when a form was submitted, who reviewed it, and what feedback was provided. This transparency allows program guarantors and teachers to understand exactly what modifications are required to move the process forward.

Technical transparency is achieved by pairing backend approval payloads

with these user-generated text descriptions. Every time a status change occurs, the system logs the `formApprovalStatusId` and links it to the corresponding message in the database. When the frontend retrieves the communication history, it merges these data points to reconstruct the audit trail. A "Rejected" status ID is paired with the approver's explanation, while a "Resubmitted" action might be paired with a note from the author detailing the corrections made.

This pairing ensures that the administrative history is both human-readable and technically accurate. Users can scroll through the communication module to see a complete narrative of the form's evolution. Because every formal action is tied to a message, the system creates a permanent and reliable record of the accreditation dialogue. This level of detail is particularly useful during the final stages of the process, as it provides a clear justification for every step taken toward final validation.

7.5.5 File-level overrides and finalization

The final stage of the accreditation process shifts focus from individual documents to the entire file. While each form has its own internal workflow, the application includes a global management layer that can override these local states. This ensures that once a file reaches a certain point in the institutional timeline, no further changes can occur regardless of whether an individual form is still in a draft or approved state. This hierarchy is essential for maintaining the integrity of the complete accreditation package before it is submitted for final review or archived.

The overarching state of the accreditation file is managed through the `useFilesStore.js` module. This store tracks the global status of the file, which is distinct from the status of the forms contained within it. Two critical states at this level are "Locked" and "Closed." A file typically enters the Locked state when the Faculty Representative decides that the preparation phase is complete and the entire package is ready for a final institutional audit. The Closed state represents the permanent conclusion of the process. Managing these states centrally allows the application to enforce rules that apply to all forms simultaneously, ensuring that the file remains a stable and unchanging record.

The locking mechanism functions as a global constraint that restricts modifications across the entire application. When a file is marked as locked, the `useFilesStore.js` triggers a state change that affects every component. The frontend interprets this global status as a priority override. Even if a specific form would normally be editable based on its own status and the user's role, the global lock forces the `isReadOnly` logic to return a true value. All "Save," "Send," and "Approve" buttons are disabled or hidden. This restriction prevents any accidental or unauthorized data entry during the final review period, guaranteeing

that the version seen by the evaluators remains exactly as it was when the lock was applied.

Closing a file is the final administrative action in the system and is subject to several technical requirements. The application does not allow a file to be closed until specific data integrity rules are met. A primary condition for this final archival is the verification of mandatory data within Form A-I, which often acts as the cover sheet for the entire accreditation. The system checks for the presence of an official approval date and other critical fields before permitting the Faculty Representative to finalize the file.

If these conditions are not satisfied, the "Close File" action remains unavailable or triggers a validation error. This logic ensures that no incomplete files are archived in the university database. Once the requirements are met and the file is closed, the system performs a final synchronization. The backend sets a terminal status that prevents any future reopening of the document, effectively preserving the accreditation file as a historical record of the institutional process.

7.6 Administration sub-application

The administration sub-application serves as the foundational management layer for the entire accreditation system. While the primary application handles the complex workflow of creating and approving accreditation files, the administration module provides the essential data required for those processes. This separation of concerns ensures that institutional information is managed centrally and consistently across all academic departments. By providing a dedicated interface for system configuration, the application allows for a high degree of flexibility without compromising the stability of the core accreditation logic.

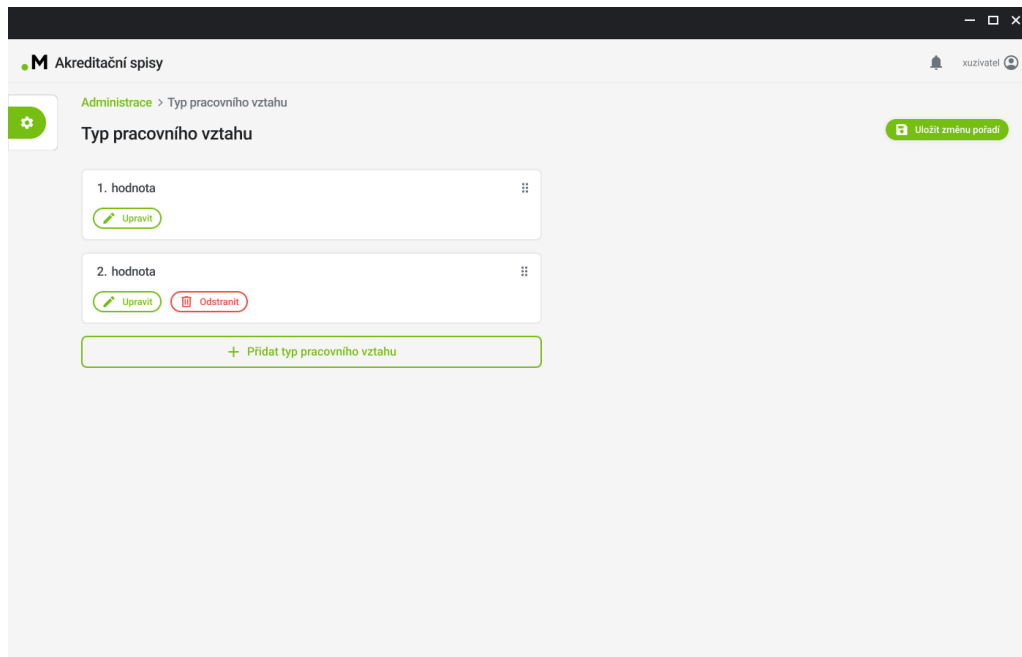
7.6.1 Purpose and role of the Administration module

The administration module functions as the system's configuration layer and acts as the central hub for all static and semi-static data. This includes institutional metadata such as study program profiles, teaching languages, and academic degree titles. Instead of hardcoding these values into the individual forms, the application retrieves them dynamically from the administration sub-application. This centralized approach guarantees that every form in the accreditation process uses the most current and officially sanctioned data. It also simplifies the management of relational data, such as ISCED codes or educational

field classifications, which must remain uniform throughout the university.

A primary technical objective of the administration module is to decouple content from the underlying application logic. By moving institutional metadata into a manageable interface, the system allows university staff to update information without requiring developer intervention or new code deployments. If the university introduces a new type of employment relationship or changes the name of a faculty, an administrator can simply update the value within the sub-application.

These changes propagate through the system immediately, allowing the main application to reflect the new data in real-time. This architectural choice significantly reduces long-term maintenance costs and empowers administrative staff to respond to regulatory changes quickly. It ensures that the software remains a tool for academic management rather than a rigid framework that requires constant technical adjustment.

**Figure 7.5**

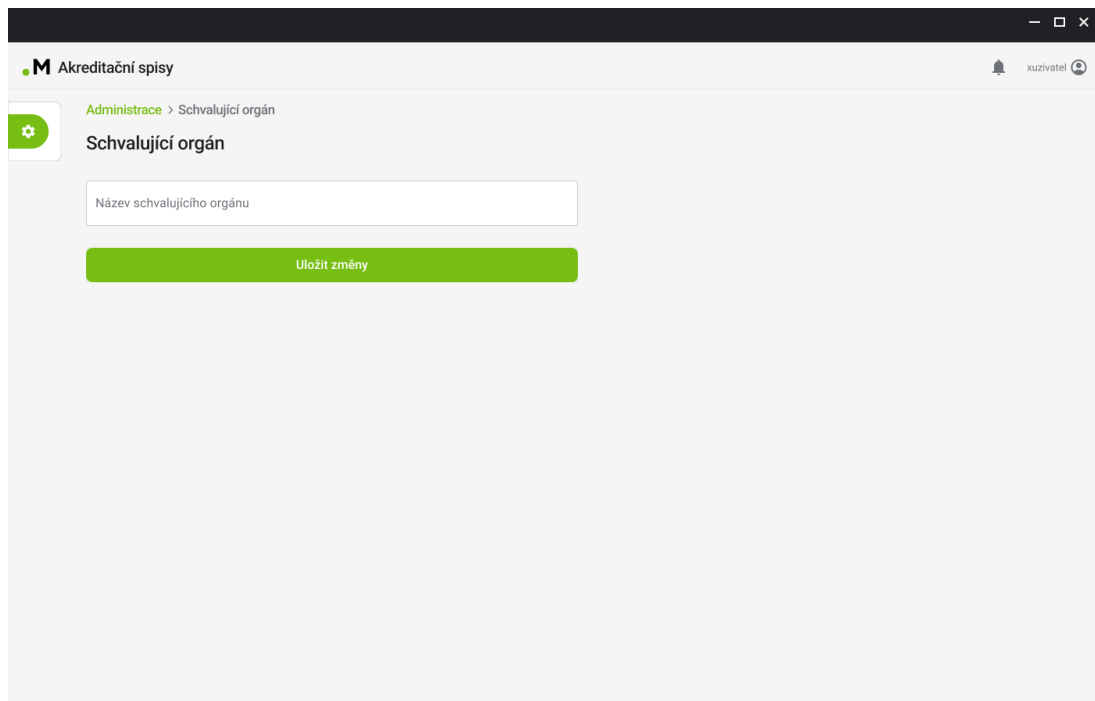
Example page with the type 1 component (Veverka, 2026)

7.6.2 Standardized component architecture

The administration sub-application is built on a modular architecture that categorizes institutional data into five distinct form types. Each type is represented by a dedicated Vue.js component designed to handle specific data structures, ranging from simple ordered lists to complex schema-driven objects. This approach ensures a consistent user experience while allowing the application to scale as new administrative requirements emerge.

Type 1: Draggable managed list

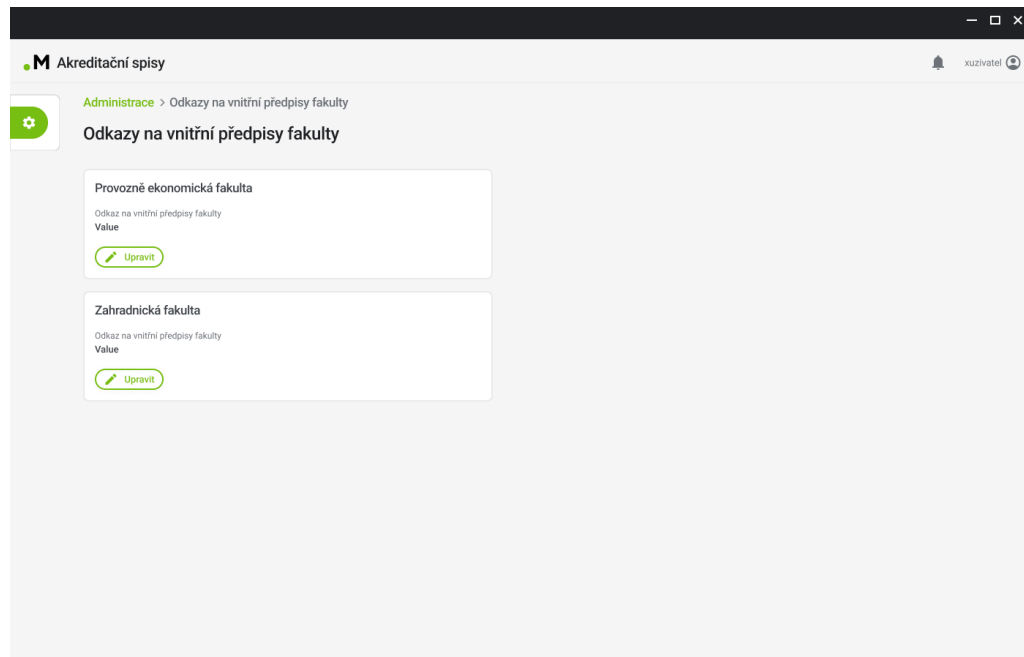
The `Type1.vue` component is utilized for entities where the specific display order is essential for the accreditation documents, such as academic degrees or study levels. It integrates the `vuedraggable` library to provide an intuitive interface for reordering items. To optimize performance, the component maintains an `initialOrderIds` reference that stores the original state of the list. When a user rearranges items, the "Save Order" button only becomes active if changes are detected, emitting an array of objects containing the updated IDs and their new numerical order. This component also supports an optional secondary display field through a `secondaryText` prop, allowing for more descriptive list items.

**Figure 7.6**

Example page with the type 2 component (Veverka, 2026)

Type 2: Flexible text configuration

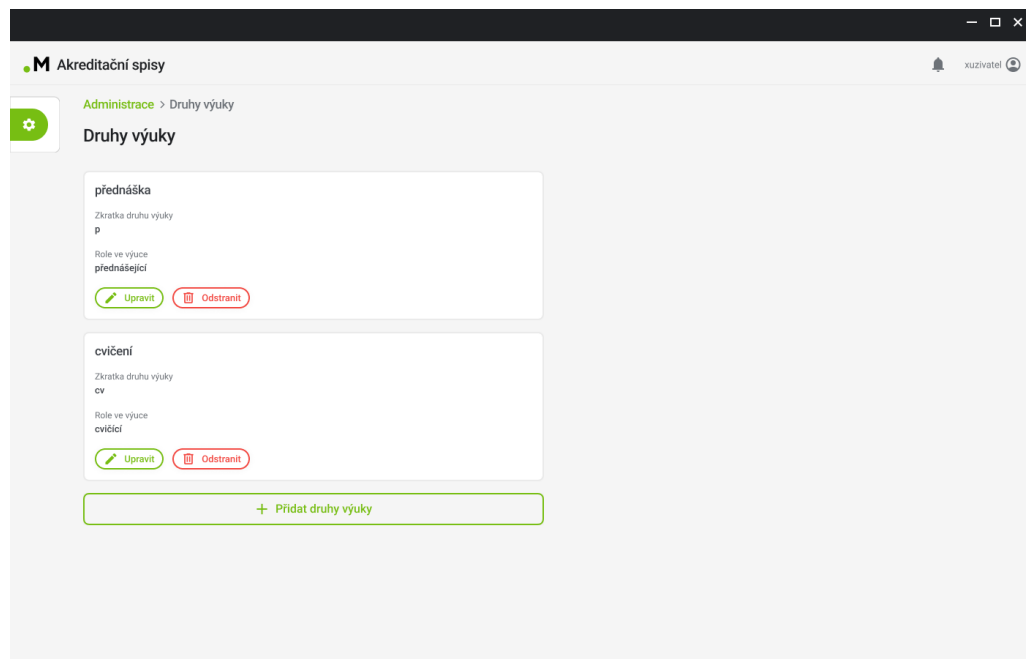
Administrative metadata consisting of single strings, links, or long-form descriptions is managed by `Type2.vue`. This component is highly flexible and adapts its interface based on the `isLarge` property. For short inputs like university names or web links, it renders a standard text field. When dealing with complex descriptions such as anti-plagiarism system measures, it switches to a `QuillEditor` to provide rich text capabilities. A minor logic layer is included to parse subtext, automatically bolding prefixes that contain a colon to improve the readability of administrative instructions.

**Figure 7.7**

Example page with the type 3 component (Veverka, 2026)

Type 3: Simple card list

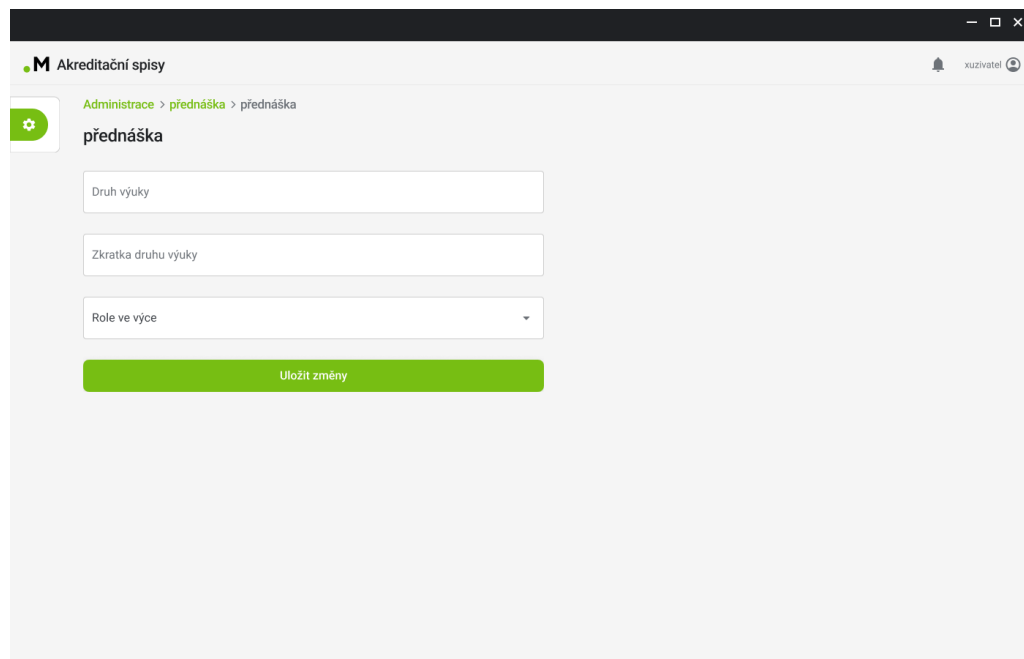
The `Type3.vue` component handles basic collections of entities that do not require manual reordering. It uses a `SmallCard` component to present each item in a clean, modular layout. The mapping of data is controlled through `valueKey` and `valueKey2` properties, which allow the parent view to specify which object attributes should appear as the card title and value. This component serves as the foundation for simple administrative lists, providing standard events for editing or removing existing items.

**Figure 7.8**

Example page with the type 4 component (Veverka, 2026)

Type 4: Enhanced card list

As a specialized extension of the Type 3 component, `Type4.vue` is designed for more complex data objects. It supports additional data mapping through a third key and a second caption field, allowing for a higher density of information within each card. Unlike the simpler version, Type 4 includes a built-in "Add Value" button at the bottom of the list. This makes it the preferred choice for managing entities that require frequent additions and more detailed descriptions, such as faculty-specific regulations.

The image shows a web browser window with a dark title bar. The page has a light gray background. At the top left, there is a logo with a green dot and the text 'Akreditační spisy'. To the right of the logo is a breadcrumb trail: 'Administrace > přednáška > přednáška'. Below the breadcrumb, there is a green gear icon in a square. The main heading is 'přednáška'. There are three input fields: the first is labeled 'Druh výuky', the second is labeled 'Zkratka druhu výuky', and the third is a dropdown menu labeled 'Role ve výce'. At the bottom, there is a wide green button with the text 'Uložit změny'.**Figure 7.9**

Example page with the type 5 component (Veverka, 2026)

Type 5: Schema-driven forms

The `Type5.vue` component is a specialized variant of the Type 2 input style, designed for entities that require multiple data fields rather than a single string. It utilizes a dynamic rendering engine that iterates through a `fields` array passed as a property. Depending on the configuration of each field, the component automatically renders the appropriate input type, such as a select menu, a textarea, or a standard text field. To maintain the integrity of the application state, the component clones the incoming data into a local `itemValue` reference. This prevents the direct mutation of properties and ensures that updates are only emitted when the user confirms their changes. This schema-driven approach is ideal for managing objects with varying structures defined by the parent administrative view.

7.6.3 Architectural design and component reusability

The administration sub-application follows a strict modular design to ensure a consistent experience for university staff regardless of the data they manage. Every administration view shares a standardized layout that typically consists of a header with action buttons and a main content area for data display. By maintaining this uniform structure, the system lowers the cognitive load for

users who must navigate between various institutional settings.

Technical efficiency is achieved through generic data management components that handle standard CRUD operations. Rather than writing unique logic for every database table, the application uses reusable wrappers that manage the fetching, editing, and deletion of records. This approach significantly reduces technical debt and ensures that bug fixes or performance improvements applied to a core component benefit the entire sub-application simultaneously.

Access to these powerful management tools is strictly controlled via route-based permissions. The application utilizes Vue Router guards to intercept navigation attempts to the administration path. The system checks the user's authenticated roles and permits entry only to System Administrators and specific Faculty Representatives. If a user without the necessary clearance tries to access these routes, the router automatically redirects them to the main dashboard, protecting sensitive institutional configurations at the architectural level.

7.6.4 Data consistency and integrity

Maintaining the integrity of accreditation records requires a specialized approach to data updates. Institutional data is not static, yet historical accreditation files must remain accurate representations of the conditions present at the time of their submission. To solve this, the application implements the "Closed File" protection rule. This logic ensures that once an accreditation file is closed, it retains its original values. Even if a global administrative setting is modified later, such as a change in a degree title, the archived file remains unchanged.

The backend supports this integrity by differentiating between open and closed files during data synchronization. When an administrator modifies a value, the backend updates all currently open accreditation drafts to reflect the new information. However, it preserves the existing state for any record linked to a finalized or closed file. This dual-track validation logic allows the university to update current standards without corrupting historical data.

On the frontend, the application uses UI alerts to manage potential data invalidity. If a user opens a draft that contains a value previously deleted or significantly modified in the administration module, a warning appears. This signals that the selected option is no longer valid for new submissions. These alerts prompt the user to review the information and select a current value, ensuring that no accreditation file is submitted with obsolete institutional data.

7.6.5 Integration with the main Accreditation workflow

The administration sub-application acts as the primary provider of information for the entire accreditation suite. Forms A-I through E do not store their own lists of academic degrees, faculties, or study modes. Instead, they utilize centralized API consumption to fetch this data from the administration endpoints. This ensures that dropdown menus and selection fields are always populated with synchronized, university-approved options.

Administrative updates are designed for real-time synchronization with the drafting process. When a change is saved in the administration module, it becomes immediately available to any user currently working on a new accreditation file. This immediate propagation ensures that the accreditation process always aligns with the latest institutional policies.

8 Testing

Testing and deployment represent the final, critical stages of the software development lifecycle. As Ramezani et al. (2025) note, these phases are essential for ensuring that an application functions correctly and remains reliable when handled by its target audience. Because the accreditation system at Mendel University processes significant amounts of academic data and must adhere to strict administrative rules, the verification process was divided into several distinct layers. This chapter, as further detailed by Veverka (2026), describes the methods used to gather user feedback, the automated testing strategy employed to ensure stability, and the final procedures for transitioning the application into a production environment.

8.1 User testing

The testing process relied heavily on direct interaction with the software to ensure it met the practical needs of the university environment. Throughout the development phase, the team performed continuous internal testing by simulating the actions of end users. This proactive approach allowed developers to identify navigation issues or interface inconsistencies early, correcting them before they could become deeply embedded in the system architecture. Such an internal feedback loop was vital for maintaining a high standard of usability as new modules were added to the client-side application.

Once a Minimum Viable Product was established, it was delivered to the customer for a preliminary round of testing by a select group of staff members. This stage was particularly valuable as it allowed for the collection of real-world feedback and the identification of bugs that might not have appeared in a controlled local development environment. The input gathered during this period was used not only to correct existing errors but also to update a backlog of future requirements and refinements to be addressed as the project evolved.

Before the official full product launch, the finished application underwent a critical testing period lasting several days. This window was vital for catching any remaining functional errors that could potentially disrupt the accreditation workflow. All critical bugs identified during this period were resolved immediately to ensure a stable and professional release. Ultimately, this thorough final check confirmed that the platform was ready for daily use and successfully met all the functional requirements established at the beginning of the project.

8.2 Front-end testing levels

Testing a modern web application requires a layered approach to ensure that individual components function correctly while the entire system remains cohesive. By categorizing tests into different levels of granularity, developers can balance the speed of execution with the thoroughness of the verification. Padolsey (2020) observes that this hierarchy allows for the early detection of bugs in isolated logic while simultaneously confirming that complex interactions across the user interface produce the expected results.

Unit testing focuses on the smallest testable parts of the software in complete isolation. In a Vue.js environment, Perdana et al. (2024) explain that these tests primarily target individual components or utility functions. The goal is to verify that a specific component renders its template correctly and reacts properly to various properties or user events. Because these tests do not rely on a live browser or external services, they provide immediate feedback to the developer. Ensuring that small UI elements—such as custom buttons or input fields—work independently is the first step toward building a resilient interface for Mendel University staff.

Integration tests move beyond isolated units to verify how different modules of the application work together. These tests confirm that data flows correctly between parent and child components and that the application state updates as intended during complex interactions. Finally, end-to-end testing represents the highest level of this hierarchy by simulating the actual journey of a user within a real browser environment. Padolsey (2020) argues that these scenarios are essential for the accreditation application because they validate the entire workflow, from the initial login to the final submission of documents. This level of testing provides the highest confidence that the software is truly ready for production.

8.3 End-to-end testing with Cypress

Automating the user journey is a critical part of maintaining the long-term reliability of the accreditation system. For this purpose, the project utilizes Cypress, a modern testing framework designed specifically for web applications. Unlike traditional testing tools that run outside of the browser, the Cypress (2026) documentation highlights that the framework operates directly within the browser environment. This unique architecture allows tests to have deep access to the application's state and provides a visual interface where developers can

monitor execution in real time.

The effectiveness of the testing suite relies on a centralized configuration that standardizes browser behavior during tests. A primary support file is used to initialize global settings and custom commands before any test case is executed. One of the most important custom commands created for this project handles the authentication process. As Talakola (2022) suggests, using a dedicated login command allows tests to avoid repeating login logic in every individual file, which makes the suite easier to maintain and significantly faster to run.

The framework is also configured to handle the practical realities of a web environment, including an exception handler that prevents tests from failing due to non-critical errors in third-party scripts. This ensures that the automated tests remain focused strictly on the actual code of the accreditation application rather than external dependencies. Together, these configurations create a stable environment where automated scripts can reliably simulate a wide range of user behaviors.

8.3.1 Implementation of automated test scenarios

The implementation of automated tests focuses on the most frequent and complex tasks performed by university staff. Three main test suites were developed to cover the core functionalities of the application and ensure a smooth user experience.

The verification of the A-I form involves a suite that checks the complete lifecycle of data entry and persistence. The test navigates to the specific form, interacts with various text fields, and utilizes a date picker to select approval dates. The test is considered successful only if the application correctly displays a confirmation message after the user clicks the save button. This process confirms that the front-end correctly communicates with backend services to store the information as intended.

The B-I form tests focus on reactive UI elements. These scenarios verify that users can successfully select options from radio groups and dynamically add or remove sections within the form. Additionally, these tests cover the scenario of clearing previously entered data. This ensures that the application state resets correctly, which prevents errors where old or irrelevant data might persist unexpectedly in the database of Mendel University.

The final suite automates the creation of an entirely new accreditation record. This process is complex because it involves several interactive components, including autocomplete fields for faculty selection and multi-select menus for choosing different study modes. The test simulates a user filling out the entire creation form and verifies that the system successfully generates the new file while displaying the appropriate success notification. This final layer of

automation ensures that the most critical entry point of the application remains functional at all times.

9 Conclusion

The digitalization of administrative agendas is an essential step for modern higher education institutions striving to maintain high standards of quality and transparency. This thesis has focused on the implementation of selected client-side functionalities for the new accreditation application at Mendel University in Brno. By replacing outdated manual workflows with a robust, web-based solution, the project successfully addresses the growing need for efficient data management within the university's internal quality assurance processes.

The initial analysis phase confirmed that the previous manual system which relied heavily on static documents and fragmented spreadsheets was no longer sustainable for a growing institution. The lack of synchronization with the University Information System (UIS) and the absence of real-time status tracking created significant administrative overhead and increased the risk of data inconsistency. These findings directly shaped the functional requirements for the new application, placing a strong emphasis on automated data retrieval, structured input validation, and a transparent, centralized approval workflow.

The practical development of this project included the implementation of core accreditation forms A-I and B-I, a centralized approval engine, and specialized administrative modules. By utilizing a technical stack based on Vue.js and JavaScript, the solution provides a reactive and user-friendly interface that simplifies complex data entry tasks. The application now enables the seamless management of accreditation files, ensuring that all participants from faculty members and program guarantors to the RVH operate within a unified and secure digital environment.

From a technical perspective, the chosen architecture proved highly effective in meeting the project's rigorous requirements. The component-based nature of Vue.js allowed for the creation of reusable and maintainable UI elements, while the integration of modern JavaScript features ensured a smooth and responsive user experience. This technical foundation provides the necessary security and performance required to handle sensitive institutional data while remaining accessible to staff members with varying levels of technical expertise.

The primary contribution of this work lies in the significant reduction of manual labor and the substantial improvement of data integrity at Mendel University. By shifting from error-prone spreadsheets to a centralized system, the university gains real-time transparency and a more reliable basis for academic reporting. Furthermore, the automation of data retrieval from the UIS not only saves time for academic staff but also ensures that accreditation dossiers are consistently based on the most accurate and up-to-date institutional records.

Looking ahead, the application has been designed with long-term extensibility

in mind. While this thesis focused on specific modules, other vital components were developed in parallel by members of the Smart MENDELU team to provide a complete toolkit for the accreditation process. To further improve developer ergonomics and system reliability, future extensions and new modules are planned to be written in TypeScript. This strategic shift will eventually lead to a full migration of the existing codebase, ensuring that the platform remains stable and maintainable for years to come.

In conclusion, the functionalities implemented as part of this thesis represent a major milestone in the broader Smart MENDELU digitalization initiative. The new accreditation application successfully bridges the gap between raw institutional data and regulatory compliance, providing a modern, efficient tool that supports the university's ongoing commitment to academic excellence and administrative transparency.

References

- ALJEHANI, A. AND QURESHI, M. R. Agile Method: Challenges and Adaptations for Complex Project Environments. *International Journal of Information Engineering and Electronic Business*, 2025, Vol. 17, p. 84–98.
- ATLASSIAN Trello. In *Getting started with Trello* [on-line]. 2024 [cit. 2026-04-20]. Available at: <https://trello.com/guide>.
- CHOUNTA, IA., ORTEGA-ARRANZ, A., DASKALAKI, S. AND OTHERS Toward a data-informed framework for the assessment of digital readiness of higher education institutions. *Int J Educ Technol High Educ*, 2024, Vol. 12, p. 59–87.
- CYPRESS Best Practices. In *Cypress docs* [on-line]. 2026 [cit. 2026-04-20]. Available at: <https://docs.cypress.io/app/core-concepts/best-practices>.
- DANIL PRISTUPOV Fork. In *Fork* [on-line]. 2025 [cit. 2026-04-20]. Available at: <https://git-fork.com/>.
- DEVOGRAPHICS State of JS. In *State of JS* [on-line]. 2026 [cit. 2026-04-20]. Available at: <https://2025.stateofjs.com/en-US/>.
- DOCKER INC. Docker. In *dockerdocs* [on-line]. 2026 [cit. 2026-04-20]. Available at: <https://docs.docker.com/>.
- GITLAB GitLabDocs. In *Find GitLab answers fast* [on-line]. 2026 [cit. 2026-04-20]. Available at: <https://docs.gitlab.com/>.
- HOU, A.Y.C., LU, I.J.G. AND HILL, C. What Has Been the Impact of COVID-19 on Driving Digitalization, Innovation and Crisis Management of Higher Education and Quality Assurance?—A Taiwan Case Study in Alignment with the INQAAHE Virtual Review. *High Educ Policy*, 2022, Vol. 35, p. 568–590. (ISSN .)
- ILIEVA, G., YANKOVA, T., RUSEVA, M. AND KABAIVANOV, S. A Framework for Generative AI-Driven Assessment in Higher Education. *Information*, 2025, Vol. 16, p. 472–493.
- ISAIFAN, R.J. Artificial Intelligence in Higher Education: A Global Statistical Synthesis for Policy and Quality Assurance Reform. *Education Sciences*, 2026, Vol. 16, p. 483–496.
- KARÁSEK, J. *Procesní optimalizace informačního systému na správu akreditací MENDELU*. (Diploma thesis.) Brno : Mendel University, 2025. 79 pp. . Supervisor: doc. Ing. Oldřich Trenz, Ph.D..
- KHOTIMAH, K. AND OTHERS. Advancing Efficiency, Transparency, and Accuracy of Digital Quality Assurance Systems in Higher Education. *Education Quarterly Reviews*, 2024, Vol. 7, No. 4, p. 300–310. (ISSN 2621-5799.)
- MENDEL UNIVERSITY Dodatek ke zprávě o vnitřním hodnocení kvality 2025. In

- Vnitřní hodnocení kvality* [on-line]. 2022 [cit. 2026-04-20]. Available at: [https://mendelu.cz/wp-content/uploads/2023/10/Akreditacni_rad_reg_11072023.pdf](<https://kvalita.mendelu.cz/vnitрни-hodnoceni-kvality>).
- MENDEL UNIVERSITY AKREDITAČNÍ ŘÁD MENDELOVY UNIVERZITY V BRNĚ ze dne 11.7.2023. In *Vnitřní předpis Mendelovy univerzity v Brně* [on-line]. 2023 [cit. 2026-04-20]. Available at: https://mendelu.cz/wp-content/uploads/2023/10/Akreditacni_rad_reg_11072023.pdf.
- MENDEL UNIVERSITY Kontrola a vnitřní hodnocení kvality akreditovaných studijních programů. In *Vnitřní předpisy univerzity* [on-line]. 2023 [cit. 2025-12-17]. Available at: https://is.mendelu.cz/auth/dok_server/slozka.pl?id=100163;download=324109.
- MENDEL UNIVERSITY *Aplikace na tvorbu formulářů akreditačního spisu - Zadání pro vyhlášení veřejné zakázky verze 2024-02-26*. 1. ed. Internal document : Oddělení kvality, 2024. - pp.
- MOSTAFAVI, Z.S. AND RAMEZANI, S.G. Internal quality assessment from theory to practice: A case study of a fully electronic university. *Educ Inf Technol*, 2026, Vol. 12, p. 440–471.
- NATIONAL ACCREDITATION BUREAU Metodické materiály pro přípravu a hodnocení žádostí o akreditaci studijního programu. In *NAÚ* [on-line]. 2024 [cit. 2026-04-20]. Available at: <https://www.nauvs.cz/index.php/cs/metodiky/71-metodicke-materialy-pro>](<https://www.nauvs.cz/index.php/cs/metodiky/71-metodicke-materialy-pro>)-pripravu-a-hodnoceni-zadosti-o-akreditaci-studijniho-programu.
- OECD *Ensuring Quality Digital Higher Education in Hungary*. 1. ed. OECD Publishing : Paris, 2023. 206 pp. ISBN ISBN 978-92-64-78605-9.
- PADOLSEY, J. *Clean Code in JavaScript: Develop Reliable, Maintainable, and Robust JavaScript*. 1. ed. Packt Publishing : UK, 2020. 548 pp. ISBN 1789957648.
- PARSONS, D., THORN, R., INKILA M. AND MACCALLUM, K. Using Trello to Support Agile and Lean Learning with Scrum and Kanban in Teacher Professional Development. *2018 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE)*, 2018, Vol. 3, p. 720–724.
- PERDANA, F. P. ., SUPRATMAN, E. ., AND SAPUTRA, D. R. Designing a Modern Web Interface with Vue.js and Tailwind for University Information System. *Brilliance: Research of Artificial Intelligence*, 2024, Vol. 4, p. 956–963.
- RAMEZANI, S.G., MOSTAFAVI, Z.S. AND GODFREY, H. A Checklist for a Comprehensive Accreditation Model for Fully Electronic Higher Education Institutions: Synthesising Criteria and Expert Perspectives. *Higher Education Quarterly*, 2025, Vol. 79, p. 36–56. (ISSN 0951-5224.)
- RAYMOND, J. Migrating a JavaScript Project to TypeScript: Challenges, Benefits, and Best Practices. *JOUR*, 2023, Vol. 5, p. 127–165.

- RIYADI S., SAKINAH, N. AND ASRONI Implementation of Vue.Js Framework for Front-End Development: A Study Case of Employee Information System. *2024 4th International Conference on Electronic and Electrical Engineering and Intelligent System (ICE3IS)*, 2024, Vol. 4, p. 105–109.
- SETIYADI, A., SUMITRA, I.D., HARDYANTO, C. AND REZA, D. Accreditation Document Digitalization System Model to Support The Validation Process by Assessors Using The Extraction Transformation and Loading Method. *Jurnal Ilmiah Komputer dan Informatika*, 2025, Vol. 14 No. 2, p. 12–20. (ISSN 2089-9033.)
- SMARTBEAR SOFTWARE Swagger. In *Swagger Documentation* [on-line]. 2026 [cit. 2026-04-20]. Available at: <https://swagger.io/docs/>.
- SONARSOURCE SÀRL SonarQube. In *Sonar Docs* [on-line]. 2026 [cit. 2026-04-20]. Available at: <https://docs.sonarsource.com/>.
- TALAKOLA, S. Exploring the Effectiveness of End-to-End Testing Frameworks in Modern Web Development. *International Journal of Emerging Research in Engineering and Technology*, 2022, Vol. 3, p. 29–39.
- TONG, J., JIKSON, R. R. AND GUNAWAN, A. A. S. Comparative Performance Analysis of Javascript Frontend Web Frameworks. *2023 3rd International Conference on Electronic and Electrical Engineering and Intelligent System (ICE3IS)*, 2023, Vol. 3, p. 81–86.
- TYLER, V. Performance Benchmarking: TypeScript vs. JavaScript in Modern Web Development. *University of Hull*, 2020, Vol. 3, p. 12–17.
- VANHEAR, J., AZZOPARDI, J., MHAMDI, C., XUERE, J., FARRUGIA, Z. AND BEZZINA, F. From Compliance to Collaboration: Redefining QA in Challenging Times through Digital Innovation. *EQUAF*, 2025, Vol. 62, p. 1–11. (ISSN 1375-3797.)
- VEVERKA, A. *Design a implementace front-endu aplikace pro tvorbu akreditačních žádostí*. (Diploma thesis.) Brno : Mendel University, 2025. 74 pp. . Supervisor: Ing. Oldřich Faldík, Ph.D..
- VEVERKA, A. Aplikace - Akreditace. In *Figma* [on-line]. 2026 [cit. 2026-04-20]. Available at: <https://www.figma.com/design/sPXIk12VjjGfYYBMrYPuMU/Aplikace---Akreditace?node-id=1805-58421&node-type=canvas&t=p2ICeEoeh6vtkqSL-0>.
- ZAKHAROVA, Y. The Concept of Digital Transformation Management in Higher Education Institutions: A PDCA-Approach to Improving Educational Quality. *European Journal of Interdisciplinary Issues*, 2025, Vol. 2, p. 50 to 56. (ISSN 3071-9658.)
- ZÁKONY PRO LIDI Zákon č. 111/1998 Sb., 29. 5. 1998, o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů. In *Zákon č. 111/1998 Sb.* [on-line]. 2026 [cit. 2026-04-20]. Available at: <https://www.zakonyprolidi.cz/cs/1998-111>.

List of Tables

4.1	Overview of functional and non-functional requirements.	26
5.1	Overview of functional and non-functional requirements.	29
7.1	Publication components.	43
7.2	Ability of a person with a certain role to modify the form based on its status.	54
7.3	Possible actions based on user role and form status.	55
7.4	Possible actions based on user role and form status.	56
7.5	Possible actions based on user role and form status.	57

List of Figures

3.1	Quality evaluation at Mendel University (Mendel University, 2022)	20
7.1	Publications section in User profile (Veverka, 2026)	44
7.2	Form A-I (Veverka, 2026)	47
7.3	Form B-I (Veverka, 2026)	51
7.4	Example file with forms with different statuses (Veverka, 2026)	53
7.5	Example page with the type 1 component (Veverka, 2026)	65
7.6	Example page with the type 2 component (Veverka, 2026)	66
7.7	Example page with the type 3 component (Veverka, 2026)	67
7.8	Example page with the type 4 component (Veverka, 2026)	68
7.9	Example page with the type 5 component (Veverka, 2026)	69